Pruning and Distilling a Mixture-of-Experts Language Model for TypeScript Specialization

Sartaj Bhuvaji [1]

Abstract

We apply the Minitron structured pruning and knowledge distillation recipe (Muralidharan et al., 2024; Sreenivas et al., 2024) to a Mixture-of-Experts (MoE) language model, Qwen3-30B-A3B-Base, and specialize the result toward TypeScript. Both Minitron papers prune only dense transformers; we extend their forward-pass-only, activation-based importance-estimation philosophy to a pruning axis neither paper covers – expert pruning. A gate-weighted expert-output-norm criterion (independently converged on by REAP (Lasby et al., 2025) and NVIDIA's Minitron-MoE extension (NVIDIA, 2024)) ranks previously-reported "Super Experts" (Su et al., 2025) first in their respective layers, and a cheap, single-forward-pass depth-pruning proxy (Block Importance) correlates strongly with the expensive, perplexity-based criterion from the original Minitron paper (Pearson $r = 0.91$; both agree on the three most critical layers). We prune, not just score, on all three axes. A depth sweep (nine layer counts, four selection strategies) shows principled selection beating random by up to 9× in relative loss once more than a handful of layers are pruned, and finds contiguous-block removal – the second Minitron paper's preferred depth-pruning approach – a mixed result on this model. An expert-pruning sweep (eight per-layer counts) shows the same pattern at a narrower margin (2–5×), and finds that pruning only the three known Super Experts costs more zero-shot loss than greedy-pruning up to 1,536 ordinary experts, reproducing a prior regression test directly in this pipeline's own metric. A head-pruning sweep (all 7 valid per-group counts under this model's grouped-query attention) shows a much narrower greedy-vs-random margin still (1.2–2.9×), and comparing all three axes at matched pruned-fractions shows experts are by far the most redundant axis on this model, with heads and depth comparably (un)forgiving to prune. Combining axes into single pruned candidates (three aggressiveness tiers, four combination types) shows real interaction effects beyond simple compounding: depth+heads combines genuinely worse than independent pruning would predict, while depth+experts is actually cheaper than independent compounding predicts. Thirteen candidates are retrained (fixed-loss knowledge distillation) and evaluated on the full 159-problem HumanEval-TS suite: experts-only pruning dominates depth-only and combined depth+experts at every matched parameter budget, no candidate beats the unpruned baseline on TypeScript specifically, and real vLLM/SGLang serving benchmarks on a subset confirm that expert pruning trades parameter count for little serving-speed cost while depth pruning trades accuracy for real per-token latency gains – two genuinely different deployment tradeoffs, not interchangeable proxies for the same thing. This is a living document, updated as further pruning, distillation, and retraining experiments complete.

1. Introduction

Minitron (Muralidharan et al., 2024) demonstrated that a large language model can be compressed into a family of smaller models via structured pruning followed by knowledge distillation, using up to 40× fewer training tokens than training each size from scratch while matching or exceeding the accuracy of from-scratch baselines. A follow-up paper (Sreenivas et al.,

[1]Seattle University, Seattle, WA, USA. Correspondence to: Sartaj Bhuvaji <sbhuvaji@seattleu.edu>.

Proceedings of the 42nd International Conference on Machine Learning, Vancouver, Canada. PMLR 267, 2025. Copyright 2025 by the author(s).

2024) applied the same recipe under a more realistic constraint – no access to the original pretraining data – and introduced teacher correction and a downstream-task-based depth-pruning criterion.

Both papers prune exclusively dense transformers, along three axes: depth, width (MLP neurons and embedding channels), and attention heads. Every weight in a dense model's MLP is applied to every token that passes through it. This is not true of a Mixture-of-Experts (MoE) model: an MoE layer routes each token to a small subset of experts (8 of 128, in the model we study here), so most of the model's parameters are never touched by any given token – and, more fundamentally, this model has no separate dense FFN for Minitron's width axis to prune at all: every one of its MLP parameters lives inside its experts. Expert pruning is therefore this model's structural stand-in for width, not a fourth axis alongside the original three, and it is the one axis neither source paper offers a formula for. This work asks what happens when the Minitron recipe is applied to a model with this property, and in particular, how to score the importance of this axis – expert pruning – that has no precedent in either source paper.

A second contribution is methodological, independent of any single axis's result. Every pruning decision in this paper – which layers, which experts, which attention heads to remove – comes from the same family of procedure: a real, measured importance signal (Block Importance or perplexity ablation for depth, a gate-weighted output-norm score for experts, an activation-based score for heads), greedily ranked and selected, never a random or positional baseline. Applying one consistent selection principle across axes, instead of a different ad hoc rule per axis, is what makes Section 11's single-axis isolation meaningful: depth24/experts0 and depth0/experts64 can be compared at a matched parameter budget (15.58B vs. 16.03B) specifically because both were chosen the same way, not because one is a tuned good candidate and the other a careless point for comparison. The same discipline extends to how results are reported: a late-caught baseline denominator mismatch (Section 4) and the finding that every candidate's accuracy was overstated at n=40, not just one surprising case (Section 10.2), are both reported and corrected rather than quietly dropped. Approaches that were tried and did not work are documented in Appendix A rather than the main narrative, so they do not interrupt it.

Our target model is Qwen3-30B-A3B-Base (Qwen Team, 2025): 48 decoder layers, 32 query / 4 key-value attention heads (GQA), 128 routed experts per layer with 8 active per token, no shared expert, approximately 30.5B total parameters of which approximately 3.35B are active for any given token. We specialize this model toward TypeScript, using bigcode/the-stack-dedup's TypeScript split (Kocetkov et al., 2022) for both calibration and retraining data, drawn from disjoint parquet shards so the two pools never overlap.

This paper is written incrementally, alongside the project's own commit history: each section is updated with real, measured results only, once the corresponding experiment has actually run. All three pruning axes – depth (Section 6), experts (Section 7), and attention heads (Section 8) – are characterized in isolation and combined with each other zero-shot (Section 9). Every candidate is then distilled against the unpruned teacher under a short, early-stopped schedule (Section 10.1) and evaluated on the full 159-problem HumanEval-TS suite, not a smaller sample (Section 10.2). Depth and experts are each isolated into a seven-candidate single-axis sweep, retrained and fully evaluated, confirming after retraining – not only zero-shot – that depth removal costs substantially more accuracy per parameter than expert removal (Section 11); attention heads are deliberately left out of this isolation, for reasons given directly (Section 11.1). Real inference-engine serving benchmarks (Section 12) then test whether that accuracy advantage also holds for serving cost, and Section 14 gives a concrete deployment recommendation.

2. Related Work

Minitron. Muralidharan et al. (2024) prune Nemotron-4 15B to 8B and 4B using purely activation-based importance estimation (no gradients), scoring attention heads, MLP neurons, and embedding channels via a shared formula that aggregates activations over a small (1024-sample) calibration set, and scoring depth separately via perplexity delta or Block Importance (cosine distance between a layer's input and output). Sreenivas et al. (2024) extend this to settings without the original pretraining data, introducing teacher correction (lightly fine-tuning the teacher on the actual distillation data before pruning) and a downstream-task-based depth-pruning criterion using contiguous-block removal, evaluated on Llama 3.1 8B and Mistral NeMo 12B.

Expert pruning in MoE models. Neither Minitron paper addresses Mixture-of-Experts models. REAP (Lasby et al., 2025) scores expert importance as a gate-weighted expert-output norm, averaged over each expert's routed tokens, and evaluates on Qwen3-30B-A3B, the same architecture family we study.

NVIDIA's Model-Optimizer library (NVIDIA, 2024) independently implements the same formula as part of its own Minitron-MoE extension, used to compress Nemotron-3-Nano-30B-A3B. Prior work on Mixtral-scale models found selection-frequency-based pruning performs worst among the criteria it tested (Lu et al., 2024), consistent with our choice not to rely on selection frequency (Section 5.2).

Super Experts. Su et al. (2025) report that Qwen3-30B-A3B contains a small number of experts (three in the first four layers of the model, by their numbering) whose rare but extremely large-magnitude outputs anchor attention sinks; ablating them alone raises WikiText-2 perplexity from 8.70 to 59.86. We treat this as a regression test for our own expert importance metric (Section 5.2).

3. Target Model and Data

3.1. Model

Qwen3-30B-A3B-Base (Qwen Team, 2025) is a base (not instruction-tuned) causal language model with 48 Qwen3MoeDecoderLayer blocks, hidden size 2048, 32 query heads and 4 key-value heads (grouped-query attention, head_dim 128), and 128 routed experts per layer (Qwen3MoeExperts, moe_intermediate_size 768) with the top 8 active per token, no shared expert. Every layer is sparse (decoder_sparse_step=1, mlp_only_layers empty). Experts within a layer are not separate submodules but two batched parameter tensors (gate_up_proj, down_proj), with the expert index as the leading dimension – pruning an expert means slicing that dimension.

3.2. Data

Calibration and retraining data are both drawn from bigcode/the-stack-dedup's data/typescript split (71 parquet shards). The first shard is reserved for calibration, the remaining 70 for retraining; because the split is shard-level rather than row-level, the two pools are disjoint by construction, which we additionally verify empirically against each sample's hexsha.

4. Baseline Evaluation of the Unpruned Teacher

Before any pruning decision, we establish the unpruned model's performance on the evaluation suite every subsequent pruning candidate will be measured against. Following both Minitron papers' evaluation settings (temperature 0.2, nucleus sampling top-p = 0.95 for generation tasks), we report:

Table 1. Baseline evaluation of Qwen3-30B-A3B-Base, unpruned.

Task	Setting	Score
HumanEval-TS	pass@1	0.6708
MBPP-TS	pass@1	0.7333
MMLU	5-shot, acc	0.8101
ARC-Challenge	25-shot, acc	0.6536
ARC-Challenge	25-shot, acc_norm	0.6749
Winogrande	5-shot, acc	0.7798

HumanEval-TS and MBPP-TS (both MultiPL-E TypeScript translations) are treated as the primary, gating tier, since they measure the capability this project specializes toward. MMLU, ARC-Challenge, and Winogrande are tracked but not optimized for, giving a comparison point against both Minitron papers' own general-knowledge tables without over-indexing on general capability at the expense of TypeScript performance.

A denominator correction, caught late. Table 1's HumanEval-TS score was measured on a 161-problem MultiPL-E TypeScript split (108/161). Every full-suite HumanEval-TS number reported anywhere else in this paper (Appendix A.1.3 onward) instead uses a 159-problem split, and this mismatch went unreconciled for over a month of project time – every comparison against the unpruned baseline in that window was silently comparing two different denominators. Caught while re-deriving exact pass@1 fractions for Section 11 below, by noticing $0.6708\overline{074}\ldots = 108/161$ does not reduce to any clean $k/159$. Re-running the identical unpruned model through the identical 159-problem harness (same temperature, top-p, batch size, no limit) gives pass@1 = 0.7170 (114/159) – the number Section 11 and every full-suite comparison from here on actually use. Both numbers stay on record rather than one silently overwriting the other: 0.6708 (108/161) is the historical Phase 1.2 result and remains in Table 1 unchanged; 0.7170 (114/159) is the number comparable to every subsequent pruned/distilled candidate in this paper.

5. Importance Estimation

Both Minitron papers score importance from forward-pass activations only, never gradients, over a small calibration set. We follow the same philosophy for every axis, including the one neither paper covers.

5.1. Attention Heads

Head importance uses the original formula unchanged. For head h in a given layer, let $a_{t,h} \in \mathbb{R}^{d_{\text{head}}}$ be that

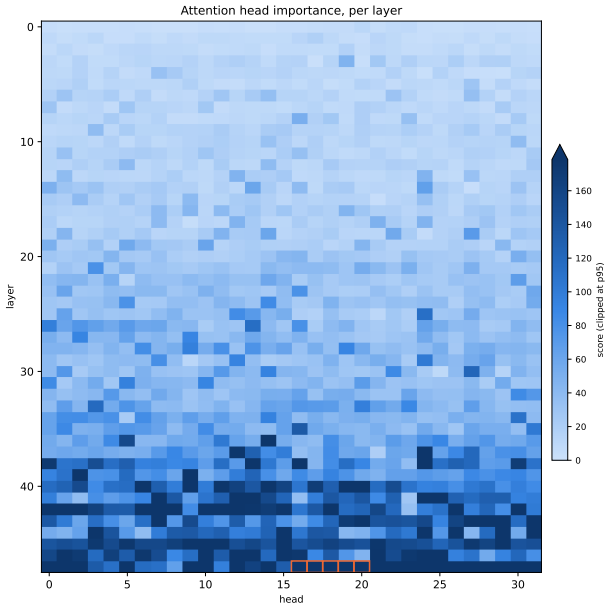


Figure 1. Head importance (F_h, Equation (1)), all 48 layers × 32 heads. Color is clipped at the 95th percentile (outlined cells mark the true top 5) since a linear scale to the true maximum leaves the rest of the grid indistinguishable from zero.

head's output for calibration token t, prior to the W^O merge. The per-token score is its L_2 norm, aggregated over every token t in the calibration set:

$$F_h = \mathrm{agg}_t \left(\|a_{t,h}\|_2 \right), \tag{1}$$

where agg is a pluggable aggregation function – Muralidharan et al. (2024) ablate mean-of-absolute-value, L_2 norm, and variance, finding the best choice is not the same for every axis; we use the L_2 variant throughout this paper. Across all 48 layers (48 calibration samples, mean length 177 tokens), scores range from 2.39 to 967.72, with the single largest value at the model's final layer (layer 47) – visible as the dark band at the bottom of Figure 1.

5.2. Experts

Neither Minitron paper offers a formula for this axis. We adopt

$$S_j = \mathrm{mean}_{x \in X_j} \left[g_j(x) \cdot \|f_j(x)\|_2 \right], \tag{2}$$

where X_j is the set of tokens routed to expert j, $g_j(x)$ is that token's renormalized routing weight, and $f_j(x)$ is the expert's output – the formula independently reached by both REAP (Lasby et al., 2025) and NVIDIA Model-Optimizer's (NVIDIA, 2024) Minitron-MoE extension, in preference to two alternatives (selection frequency alone; router logit magnitude alone) both cited sources report underperforming it (see also Section 2's note on selection-frequency-based pruning more broadly).

Algorithm 1 Expert importance from routing data

Input: expert indices $e_1, \ldots, e_N$, gate weights $g_1, \ldots, g_N$, expert outputs $f_1, \ldots, f_N \in \mathbb{R}^d$ for every (token, selected-expert) pair in the calibration set; number of experts E
$\mathrm{sum}[1..E] \leftarrow 0$; $\mathrm{count}[1..E] \leftarrow 0$
for $i = 1$ to N **do**
  $\mathrm{sum}[e_i] \leftarrow \mathrm{sum}[e_i] + g_i \cdot \|f_i\|_2$
  $\mathrm{count}[e_i] \leftarrow \mathrm{count}[e_i] + 1$
end for
for $j = 1$ to E **do**
  $S_j \leftarrow \mathrm{count}[j] > 0 \ ? \ \mathrm{sum}[j]/\mathrm{count}[j] \ : \ 0$
end for
Return: $S_1, \ldots, S_E$

Unlike head importance, there is no dense batch/sequence grid to aggregate over: routing is sparse and data-dependent per token, so S_j is computed from a flat list of (token, expert) assignments rather than a fixed-shape tensor, mirroring how the model's own expert-dispatch loop iterates (Algorithm 1).

An expert with zero routed tokens in the calibration set scores 0 and ranks lowest by construction – appropriate for a calibration-set-specific score, but a real risk if a rare-but-critical expert happens to be undersampled by chance, which is exactly what the Super Experts check below guards against.

As a validation check, not a designed experiment: all three Super Experts reported by Su et al. (2025) for this exact model (layer 1 / expert 68, layer 2 / expert 92, layer 3 / expert 82) rank first among all 128 experts in their respective layers under this metric, on real calibration data, at two different sample sizes (24 and 48 samples) – the three isolated bright cells in Figure 2. On average across all 48 layers, the top ∼16% of experts (20 of 128) already account for 33% of a layer's total importance mass (Figure 3), indicating substantial redundancy among the remaining experts as a candidate for pruning.

5.3. Depth: Block Importance and Perplexity

We implement both depth-scoring criteria from Muralidharan et al. (2024). Block Importance (BI) for layer ℓ, with input activations X_ℓ and output activa-

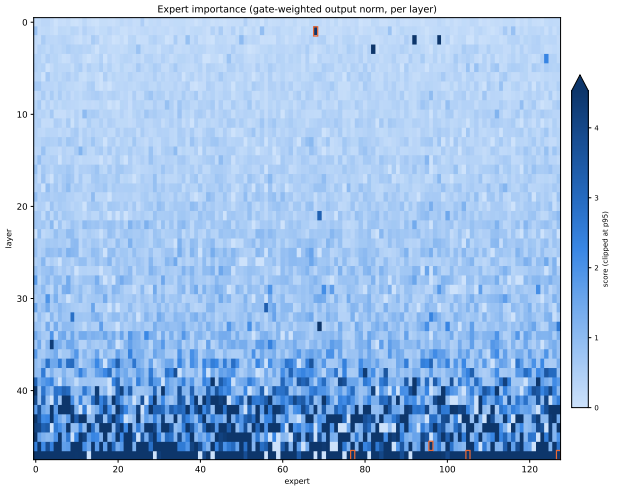


Figure 2. Expert importance (S_j), all 48 layers × 128 experts, same clipping convention as Figure 1 (outlined cells mark the true top 5 model-wide, not per-layer). Only one Super Expert (Su et al., 2025) (layer 1, expert 68) is among them; the other two rank highest only within their own layer, not model-wide – but the 95th percentile color clip still renders all three as visually saturated, since each is well above that threshold.

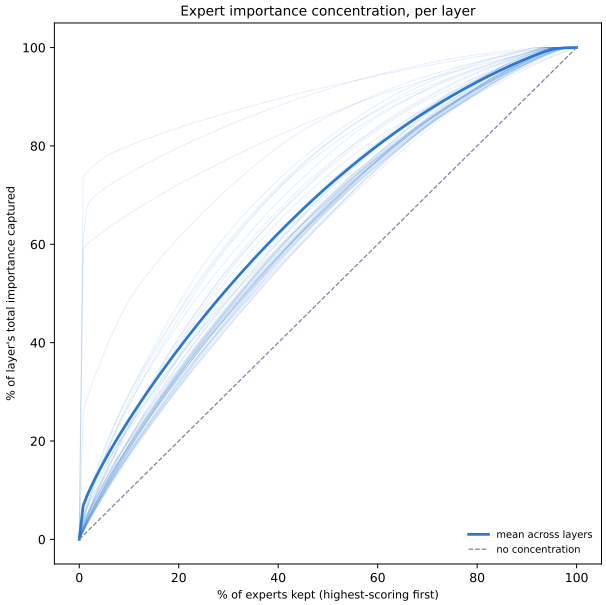


Figure 3. Per-layer cumulative importance captured by the top-k highest-scoring experts (thin lines: all 48 layers; bold: mean). The dashed diagonal is the no-concentration reference.

tions $X_{\ell+1}$ over calibration tokens t, is

$$\mathrm{BI}_\ell = 1 - \mathbb{E}_t\left[\frac{X_{\ell,t}^\top X_{\ell+1,t}}{\|X_{\ell,t}\|_2 \|X_{\ell+1,t}\|_2}\right], \tag{3}$$

computable in the same forward pass that produces the two activations. The perplexity-delta criterion instead removes one layer at a time and measures the resulting change in held-out loss relative to the unmodified baseline:

$$\Delta_\ell = \mathcal{L}(\theta \setminus \ell) - \mathcal{L}(\theta), \tag{4}$$

where $\mathcal{L}$ is mean cross-entropy loss over the calibration set and $\theta \setminus \ell$ denotes the model with layer ℓ temporarily skipped (Algorithm 2). This costs $L+1$ forward passes for an L-layer model, versus BI's one, and is run on a smaller calibration sample (16 samples, vs. 48 for BI) for that reason.

Both criteria independently identify the same pattern: the first and last layers of the model matter far more than the middle two-thirds (Figure 4). BI's highest score is at layer 0 (0.488) and second-highest at layer 47 (0.415); its lowest is layer 12 (0.011). The perplexity criterion shows the same edges-matter-most shape at a far more extreme scale – layer 0's removal costs 1.99 nats of cross-entropy, next-highest is layer 47 at 0.60 nats, and the lowest (layer 14) costs only −0.0003 nats.

Across all 48 layers, the Pearson correlation between BI and the perplexity delta is $r = 0.91$ (Figure 5),

Algorithm 2 Perplexity-delta depth scoring

Input: model θ with layers $1, \ldots, L$; calibration batches $\mathcal{D}$
base $\leftarrow$ MeanLoss$(\theta, \mathcal{D})$
for $\ell = 1$ to L do
  replace layer ℓ's forward with the identity function
  $\Delta_\ell \leftarrow$ MeanLoss$(\theta, \mathcal{D})$ − base
  restore layer ℓ's original forward function
end for
Return: $\Delta_1, \ldots, \Delta_L$

and the two criteria pick the identical top-3 most critical layers (0, 47, 1). Given the 49× cost difference between the two, BI is a reliable cheap stand-in for perplexity-based scoring when iterating over many candidates, reserving the expensive criterion for confirming a final choice.

Figure 6 places the three cheap, single-forward-pass methods – heads, experts, BI – on the same (min-max normalized) axis; Figure 5 above already gives the fourth, perplexity, directly against BI. Heads and experts agree closely with each other throughout the model; BI diverges from both through the middle third, agreeing with them only at the two edges – the same edges where Figure 5 shows it also agrees with perplexity.

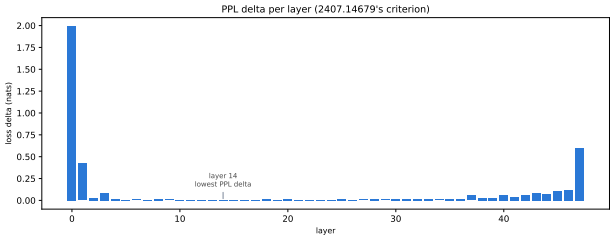


Figure 4. Perplexity delta Δ_ℓ (Equation (4)) per layer.

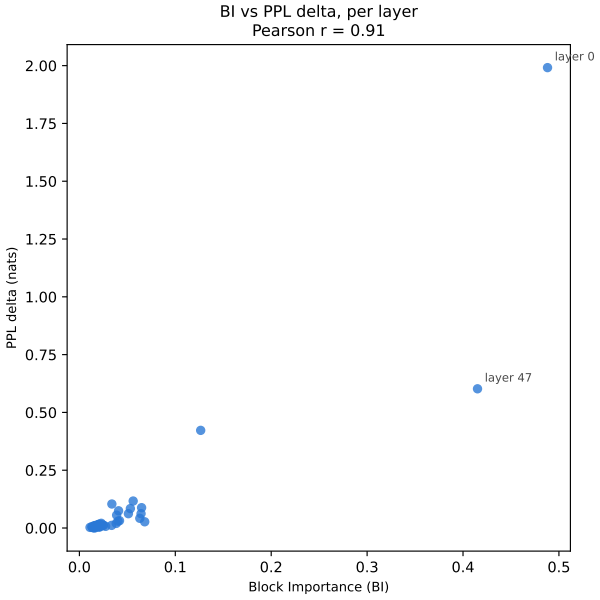


Figure 5. BI (Equation (3)) vs. perplexity delta (Equation (4)), one point per layer.

6. Depth Pruning: Zero-Shot Loss vs. Layers Pruned

Sections 5.2 and 5.3 only score which components matter; this section actually prunes them. bonsai.pruning.depth.prune_layers permanently removes a chosen set of decoder layers – unlike skip_layer's temporary identity-swap used for scoring – and renumbers each surviving layer's attention-cache index to match its new position, since the default KV cache indexes by position and breaks on the non-contiguous indices removal otherwise leaves behind. bonsai.importance.depth.mean_loss measures the resulting model's zero-shot loss, here on 32 fresh calibration samples (seed 1, a disjoint draw from every prior calibration slice in this paper).

We sweep nine layer counts ($n \in \{1, 2, 4, 6, 8, 12, 16, 20, 24\}$ of 48) under four selection strategies: bi_greedy (the n lowest-BI layers,

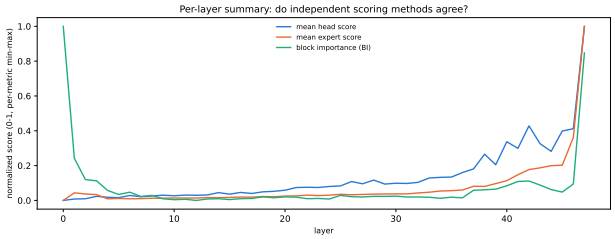


Figure 6. Mean head score, mean expert score, and BI, per layer, min-max normalized within each method.



Figure 7. Zero-shot loss vs. layers pruned. Random shown as its 3-seed mean (dashed) and min-max band. Log-scale y-axis.

Equation (3)); ppl_greedy (the n lowest-perplexity-delta layers, Equation (4)); contiguous_bi (the single contiguous block of n layers with the lowest summed BI – testing Sreenivas et al. (2024)'s own claim, citing Gromov et al., that contiguous removal beats scattered removal); and random (n random layers, 3 seeds), the control every principled strategy has to beat. Each candidate costs one forward pass, since the ranking is already decided.

Principled selection clearly beats random at every count tested. In relative-loss-increase terms (Table 2), the gap approaches an order of magnitude as n grows: at $n = 16$, bi_greedy is +41.6% over baseline versus random's +372.6% mean (9.0×), and at $n = 24$, bi_greedy/ppl_greedy are +153.8%/+135.6% against random's +1000.1% mean (6.5–7.4×; in raw loss, 2.45/2.28 against a range of 4.81–14.65 across the three seeds). bi_greedy and ppl_greedy track each other closely at every count – not merely in the ranking correlation already established ($r = 0.91$, Section 5.3) but in the actual pruning outcome – reinforcing BI as the working criterion for iterating on candidates.

Contiguous-block removal is a genuinely mixed result on this model, not a clean win either way. It is com-

Table 2. Loss increase over baseline (0.9670), by strategy and layer count.

n	bi_gr.	ppl_gr.	contig.	rand. (mean)
1	+0.3%	+0.9%	+0.3%	+5.3%
2	+1.8%	+1.4%	+0.9%	+7.5%
4	+4.3%	+3.8%	+3.6%	+14.5%
6	+8.1%	+7.0%	+7.3%	+31.7%
8	+12.0%	+11.9%	+11.3%	+40.7%
12	+21.9%	+22.2%	+24.6%	+101.8%
16	+41.6%	+46.4%	+44.5%	+372.6%
20	+76.7%	+73.5%	+76.2%	+568.8%
24	+153.8%	+135.6%	+211.0%	+1000.1%

petitive – marginally best – at small n ($n = 2$: +0.9% vs. bi_greedy's +1.8%), but falls behind both greedy strategies as n grows, most visibly by $n = 24$ (+211.0% vs. +135.6–153.8%). A plausible reading: BI's lowest-scoring layers already cluster in a contiguous run in the middle of the model (Figure 6), so for small n the two approaches pick nearly the same layers anyway; forcing strict contiguity only starts costing something once n exceeds what that natural cluster contains, pushing the contiguous search into a wider, less-optimal window.

7. Expert Pruning: Zero-Shot Loss vs. Experts Pruned

Section 6 pruned layers; this section prunes experts. bonsai.pruning.experts.prune_experts permanently slices the chosen experts out of Qwen3MoeExperts's gate_up_proj and down_proj and the matching rows of Qwen3MoeTopKRouter's weight matrix, per layer. Unlike the layer-index renumbering Section 6 needed, no such step is required here: the model re-derives which experts were hit fresh from routing indices on every forward pass rather than depending on a fixed per-expert identity, so a pruned expert's neighbors need no bookkeeping to stay consistent. We measure the resulting zero-shot loss the same way as Section 6, on the same 32 fresh calibration samples.

We sweep eight per-layer expert counts ($n \in \{8, 16, 32, 48, 64, 80, 96, 112\}$ of 128), pruned uniformly across all 48 sparse layers, under two selection strategies: greedy (the n lowest-S_j experts per layer, Equation (2), with the three Super Experts identified in Section 5.2 exempted from the ranking entirely rather than merely checked afterward); and random (n random experts per layer, same exemption, 3 seeds). We additionally evaluate one candidate outside the count sweep – super_experts_only – that prunes just the three known Super Experts and nothing else, to reproduce Su et al. (2025)'s own regression test directly in this pipeline's loss metric rather than only by citation.



Figure 8. Zero-shot loss vs. experts pruned per layer. Random shown as its 3-seed mean (dashed) and min-max band; the horizontal reference line is the super_experts_only candidate. Log-scale y-axis.

Table 3. Loss increase over baseline (0.9670), by strategy and per-layer expert count. "total" is experts removed across all 48 layers.

n	total	greedy	random (mean)
8	384	0.0%	+7.7%
16	768	+3.0%	+19.0%
32	1536	+9.6%	+43.0%
48	2304	+13.7%	+71.8%
64	3072	+25.5%	+105.9%
80	3840	+46.3%	+167.4%
96	4608	+83.2%	+256.4%
112	5376	+229.7%	+434.0%

Greedy selection beats random at every count tested (Table 3), though by a narrower margin than depth pruning's: roughly 2–5× in relative-loss terms here, versus close to an order of magnitude for layers (Section 6). A plausible reading is that 128 experts per layer offers far more substitutable choices than 48 layers does, so principled selection matters less dramatically even though it still matters clearly. Pruning is close to free up to a point: $n = 8$ (384 experts total, across all layers) lands at the same loss as the unpruned baseline (0.9670); the curve only bends noticeably past $n = 48$–64.

The starkest finding in this sweep is not on the main curve at all: pruning only the three known Super Experts – 3 of the model's 6,144 experts – costs more zero-shot loss (+51.8%) than greedy-pruning up to 1,536 ordinary experts ($n = 32$, +9.6%). Greedy selection does not cross the super_experts_only line until somewhere between $n = 80$ and $n = 96$ (3,840–4,608 experts), roughly 1,300–1,500× as many experts as the three Super Experts alone. This reproduces Su et al. (2025)'s claim (isolated ablation of these experts alone raised WikiText-2 perplexity from 8.70 to 59.86) di-

rectly in this pipeline's own loss metric and on this project's own TypeScript calibration data, not merely by citation, and is the concrete justification for exempting these experts from the ranking (Section 5.2) rather than relying only on the score itself to never rank them low.

8. Head Pruning: Zero-Shot Loss vs. Query Heads Pruned

Section 7 pruned experts; this section prunes attention heads. bonsai.pruning.heads.prune_heads permanently slices the chosen query heads out of Qwen3MoeAttention's q_proj (output rows) and o_proj (input columns) at head boundaries. Unlike the other two axes, heads cannot be selected freely: Qwen3-30B-A3B-Base's grouped-query attention (GQA) shares every 8 consecutive query heads onto one key/value head (num_key_value_groups = 8, Section 3), and Qwen3MoeAttention.forward reads that value fresh off the module on every call via repeat_kv – pruning a ragged count per group would leave it undefined. We therefore prune the same count of query heads from every key/value group in a layer, never an entire group (which would orphan its key/value head), and recompute self_attn.num_key_value_groups for the new count. We measure the resulting zero-shot loss the same way as Sections 6 and 7, on the same 32 fresh calibration samples.

This constraint fixes the sweep's entire valid search space rather than leaving a larger one to sample from, unlike the other two axes: group size 8 admits exactly 7 per-group counts ($k \in \{1, \ldots, 7\}$, since $k = 8$ would orphan a key/value head), so we sweep all of them under two selection strategies: greedy (the k lowest-F_h-scoring query heads per group, Equation (1)); random (k random heads per group, 3 seeds). This run used a rented 4x A6000 instance via device_map="auto" sharding rather than the single 80GB GPU the other two sweeps used, since no H100/A100-80GB capacity was available at launch time.

Greedy selection beats random at every count tested (Table 4), but by a much narrower margin than either other axis, and one that shrinks as k grows: 2.7× at $k = 1$ down to 1.2× at $k = 7$, versus depth's up to 9× (Section 6) and experts' 2–5× (Section 7).

The narrowing greedy/random gap raises an obvious question: is this model's attention-head axis simply more redundant than depth or experts, so any reasonable subset survives about as well as any other – or is it the least redundant, unable to absorb pruning gracefully regardless of which heads are chosen? Comparing absolute cost at every pruned-fraction this sweep shares with the other two answers it. Depth's counts coincide with heads' at 3 fractions (12.5%, 25%, 50%); experts' counts coincide at all 7. At every shared point, expert pruning is dramatically cheaper than either heads or depth – e.g. at 50% pruned, greedy costs +25.5% for experts versus +129.6% for heads and +153.8% for depth. Heads and depth track each other closely at all 3 points they share (12.5%: +5.2% vs. +8.1%; 25%: +21.4% vs. +21.9%; 50%: +129.6% vs. +153.8%), with depth's greedy loss slightly higher every time – a small, consistent difference on a single-seed, 32-sample run, not one we lean on as a strong ranking claim. The robust reading is that experts are far more redundant than either heads or depth on this model; heads and depth look comparably (un)forgiving to prune, and the narrowing greedy/random gap on heads reflects that low redundancy – with few good substitutes available in a group of 8, even a bad (random) choice is not that much worse than a good one, not because head choice stops mattering.

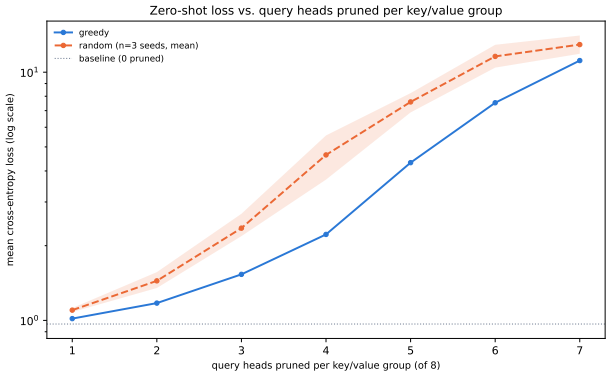


Figure 9. Zero-shot loss vs. query heads pruned per key/value group. Random shown as its 3-seed mean (dashed) and min-max band. Log-scale y-axis.

Table 4. Loss increase over baseline (0.9668), by strategy and per-group head count. "total" is query heads removed across all 48 layers.

k	total	greedy	random (mean)
1	192	+5.2%	+13.8%
2	384	+21.4%	+49.3%
3	576	+58.5%	+143.4%
4	768	+129.6%	+380.0%
5	960	+346.8%	+684.9%
6	1152	+678.9%	+1097.5%
7	1344	+1052.1%	+1236.4%

Table 5. Per-axis target counts, by tier – depth (n of 48 layers), experts (n of 128/layer), heads (k of 8/group). % is that axis's own single-axis zero-shot loss increase at that count (Sections 6–8).

tier	depth	experts	heads
conservative	n=8 (+12.0%)	n=48 (+13.7%)	k=1 (+5.2%)
moderate	n=12 (+21.9%)	n=64 (+25.5%)	k=2 (+21.4%)
aggressive	n=16 (+41.6%)	n=80 (+46.3%)	k=3 (+58.5%)

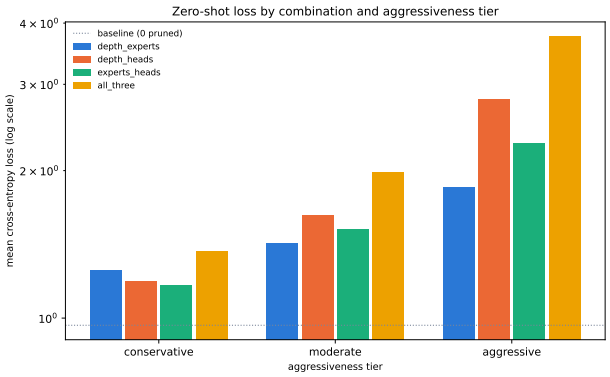


Figure 10. Zero-shot loss by axis-combination and aggressiveness tier. Log-scale y-axis.

9. Combining Axes: Depth, Experts, and Heads Together

Sections 6–8 each prune one axis at a time; this section combines them into single candidates. Per-axis target counts are chosen so each axis's own single-axis zero-shot cost lands in a similar band, not a uniform pruned fraction across axes – Section 8 already showed experts are far more redundant than depth or heads, so a uniform fraction would leave experts badly underexploited. Three aggressiveness tiers result:

For each tier, we build and measure four combinations – depth+experts, depth+heads, experts+heads, and all three – by applying prune_layers, prune_experts, and prune_heads in sequence (depth first, when present) on the same model, then measuring zero-shot loss the same way as the single-axis sweeps. Combining depth pruning with experts and/or heads requires one correctness step the single-axis sweeps didn't: expert_scores/head_scores are indexed by the original 48-layer model, so after depth pruning removes layers, the expert/head selection has to be computed against only the surviving layers' scores, in their new post-depth-pruning order – including remapping the Super Experts protected set (Section 5.2) to whichever of layers 1/2/3 actually survive.

all_three is the worst zero-shot candidate at every tier,

Table 6. Loss increase over baseline (0.9668), by tier and combination.

tier	depth+exp.	depth+heads	exp.+heads	all three
conservative	+29.4%	+22.9%	+20.8%	+41.6%
moderate	+46.9%	+67.8%	+57.0%	+105.2%
aggressive	+91.1%	+189.3%	+135.2%	+288.7%

Table 7. Actual loss increase minus the naive-multiplicative prediction (percentage points), by tier and combination.

tier	depth+exp.	depth+heads	exp.+heads	all three
conservative	+2.1pp	+5.1pp	+1.2pp	+7.7pp
moderate	−6.1pp	+19.8pp	+4.7pp	+19.5pp
aggressive	−16.0pp	+64.9pp	+3.4pp	+60.4pp

as expected from combining the most axes – but Muralidharan et al. (2024)'s own finding (depth/width rankings can flip after retraining) means this is not grounds to drop it before a real lightweight retrain (Section 13) actually tests that.

9.1. Do axes compound independently, or interact?

If axes contributed independent, uncorrelated damage, a combined candidate's relative loss increase should track the naive multiplicative prediction: $(1+a)(1+b)\cdots - 1$, where $a, b, \ldots$ are each axis's own single-axis relative loss increase at that tier's count (Table 5). Comparing that prediction to what we actually measured reveals real interaction effects, not just "more pruning costs more":

depth+heads compounds genuinely worse than independent at every tier, and the excess grows sharply with aggressiveness. depth+experts is actually subadditive at the moderate and aggressive tiers – cheaper than independent compounding predicts. experts+heads tracks the naive prediction closely throughout (+1.2pp to +4.7pp), the closest thing to genuine independence among the three pairwise combinations tested.

A plausible reading, not a proven one: depth pruning removes whole layers, including their attention heads entirely. Combining it with separate head pruning on the surviving layers may compound damage to the same mechanism (attention) rather than spreading the cut across more orthogonal components the way depth+experts does (removing layers alongside MLP/expert capacity on the survivors hits more independent parts of the model). Consistent with this account: experts are also the single most redundant axis by a wide margin (Section 8), so depth+experts is the cheapest or near-cheapest pairwise combination at every tier, while depth+heads is the most expensive

pairwise combination at every tier.

These are zero-shot numbers only. The candidate-selection goal – a TypeScript-specialized model that is smaller, faster, and cheaper to serve than the unpruned baseline, and high-accuracy on TypeScript specifically – needs real retraining and real eval-suite numbers (Section 13), not zero-shot loss alone. This sweep narrows the candidate space and surfaces a real interaction effect worth reporting either way, not a final ranking.

We distill every candidate in this paper against the unpruned teacher's own cached logits, not a fine-tuned ("corrected") teacher. Teacher correction, following Sreenivas et al. (2024), was tested in two configurations (full-parameter and LoRA) and found to regress HumanEval-TS accuracy rather than help, despite improving validation loss in both cases (Appendix A). Reaching this recipe also required finding and fixing two bugs in the knowledge-distillation loss itself, after several independent candidates and training-recipe changes all produced the same repetition-collapse failure on HumanEval-TS; the diagnosis and fix are reported in Appendix A, not here, since the loss implementation used for every result below is already the corrected one.

10. Distillation Retraining and the Full Candidate Sweep

This section covers the full candidate sweep under the final recipe: knowledge distillation against the uncorrected teacher's cached top-20 logits, a short, early-stopped schedule (established below), and full re-evaluation on the 159-problem HumanEval-TS suite rather than a 40-problem sample.

10.1. A Short, Early-Stopped Schedule

Validation loss on this recipe peaks well before a 3000-step budget completes (step 199–259) and measurably overfits past that point (Appendix A has the full derivation). We test a 500-step schedule (75-step warmup, the same 15% warmup fraction, identical peak LR and optimizer) with more frequent validation checks, on two candidates: the aggressive depth+experts candidate (8.48B, 27.8%) and the experts-only isolation candidate at a matched parameter budget (depth0/experts97, 8.55B, 28.0%).

Wall-clock drops from ∼2h20m (3000 steps) to ∼31–39 minutes – roughly 3.6–4.6× faster – while landing in the same n=40 ballpark as depth12/experts80's own 3000-step best (0.55, Appendix A.2.12), not obviously worse for using a sixth of the steps.

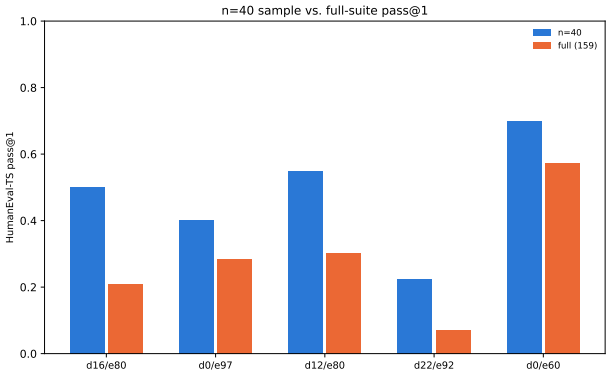


Figure 11. n=40 vs. full-suite pass@1, the same five candidates as Table 9.

(Depth12/experts80 still edges both candidates here, plausibly because it keeps more parameters, 31.0% vs. 27.8–28.0%, rather than anything about the schedule.) The shorter schedule recovers comparable downstream accuracy at a fraction of the compute, resolving Section 13's open question on this point – adopted as the standard recipe for every remaining candidate in this paper.

Two further candidates test the same schedule at opposite ends of an accuracy-vs-size curve: a large experts-only cut, depth0/experts60 (16.94B, 55.5%, zero-shot loss 0.959), and a deep combined cut, depth22/experts92 (5.53B, 18.1%, zero-shot loss 2.496). Depth0/experts60 distills well: validation loss best 0.136 at step 49 (final 0.204 at 499, a real early peak this time), n=40 pass@1 0.70 best / 0.675 final. Depth22/experts92's much worse zero-shot surface does not recover as well: validation best = final 0.513 at step 499, n=40 pass@1 0.225. Smaller is not better on this pair – consistent with Appendix A.2.6's earlier "a lighter cut" framing – though depth0/experts60's full-parameter KD needed a GPU with more than 80GB of memory, a real cost the n=40 number alone does not capture.

10.2. Full-Suite Re-Evaluation: n=40 Systematically Overstates Accuracy

Every candidate above is measured on a fixed 40-problem sample, the same sample used throughout this paper for iteration speed. We re-evaluate all five on the full 159-problem suite, no –limit, identical harness settings.

Every single candidate's full-suite score is substantially lower than its n=40 score – not just depth0/experts60, which earlier notes in this project's working log had flagged as a single surprising case. The relative

Table 8. Short-schedule (500-step) fixed-loss KD, n=40 HumanEval-TS. Both candidates' validation minimum lands on the final step itself – no separate early-stop checkpoint.

Candidate (∼params)	Zero-shot loss	KD val (step)	n=40
Aggressive, depth16/e80 (8.48B)	1.848	0.338 @ 499	0.50
Depth0/experts97 (8.55B)	1.322	0.382 @ 499	0.40

Table 9. n=40 vs. full-suite (159-problem) HumanEval-TS pass@1, same checkpoints. Every candidate overstates at n=40, by an inconsistent margin.

Candidate	n=40	Full (159)	Relative overstatement
Aggressive, depth16/e80	0.50	0.208 (33/159)	+140%
Depth0/experts97	0.40	0.283 (45/159)	+41%
Depth12/experts80 (raw-file best)	0.55	0.302 (48/159)	+82%
Depth22/experts92	0.225	0.069 (11/159)	+226%
Depth0/experts60 (best ckpt)	0.70	0.572 (91/159)	+22%

overstatement ranges from +22% to +226%, and is not simply explained by how aggressively a candidate is pruned: the three candidates that keep the most similar fraction of parameters (27.8–31.0%, aggressive/depth0-experts97/depth12-experts80) span +41% to +140% among themselves, a wider spread than their pruning fractions would suggest. The direction of the bias is consistent – n=40 never understates – but its magnitude is not reliably predictable from the n=40 number alone, which has a direct methodological consequence: a confirmatory full-suite run cannot be skipped for any candidate, including ones whose n=40 score looks unremarkable. Every n=40 number elsewhere in this paper should be read as a provisional triage signal, not a final one – it correctly ranked candidates relative to each other in every case checked here, but not their absolute accuracy.

11. Isolating the Pruning Axes After Retraining

Every full-suite candidate in Section 10 combines depth and experts in one step (or, for depth0/experts60 and depth0/experts97, prunes experts alone at zero depth). No result yet isolates each axis's own contribution across a real range after retraining – Appendix A.2.5 asked this question zero-shot, at a single matched parameter budget, but never followed it through distillation and a full-suite eval. This section fills that gap on two axes, and explains why a third – heads – is deliberately left out.

11.1. Why Not Heads

Section 8 characterizes head pruning zero-shot and Section 9 includes it in the three-axis combined sweep, but no head-only or head-combined candidate was carried into retraining and full-suite evaluation alongside the depth-only and experts-only candidates below. Three findings, all already established earlier in this paper, motivate that choice rather than a late one: heads are not cheap to prune (at matched 50%-pruned-fraction, greedy head pruning costs +129.6% zero-shot loss, comparable to depth's +153.8% and nowhere near experts' +25.5%, Section 8); heads barely reduce model size (query heads are a small fraction of total parameters in this MoE model, dominated by the per-layer expert MLPs, Section 5.2 – even an aggressive per-group head cut moves total parameter count very little); and combining heads with depth compounds unfavorably, not favorably (Section 9's interaction-effects finding: depth+heads costs more than the sum of its independent zero-shot costs, unlike depth+experts, which compounds better than independent). Given a head-only axis that is simultaneously expensive to prune and ineffective at shrinking the model, the GPU budget this section spends instead isolates depth and experts – both of which show far more favorable cost curves, and depth in particular directly extends the "is depth specifically the problem" question Appendix A.2.5 raised but never settled after retraining.

11.2. Seven Single-Axis Candidates

We build, distill (Section 10.1's short fixed-loss schedule), and fully evaluate five pure experts-only candidates (depth_n=0, $\text{expert}_n \in \{112, 64, 32, 16, 8\}$) and two pure depth-only candidates (expert_n=0, $\text{depth}_n \in \{16, 24\}$), chosen to bracket the depth cuts already measured in combined candidates (12, 16, 22 layers).

11.3. Depth Costs More Per Parameter Than Experts

The experts-only axis in Table 10, together with depth0/experts60 from Section 10.1 (16.94B, also depth_n=0, full pass@1 0.572), spans six points from barely-pruned (experts8, 94% of experts kept)

Table 10. Seven single-axis candidates, fixed-loss short KD, full 159-problem HumanEval-TS. Unpruned baseline (Section 4, reconciled denominator): pass@1 = 0.717 (114/159).

Candidate	∼params	Zero-shot loss	KD val (step)	Full pass@1
depth0/experts112	5.15B	2.567	0.569 @ 499	0.082 (13/159)
depth0/experts64	16.03B	0.970	0.145 @ 39	0.579 (92/159)
depth0/experts32	23.28B	0.889	0.070 @ 29	0.604 (96/159)
depth0/experts16	26.91B	0.874	0.047 @ 29	0.648 (103/159)
depth0/experts8	28.72B	0.869	0.042 @ 29	0.704 (112/159)
depth16/experts0	20.56B	1.157	0.207 @ 49	0.283 (45/159)
depth24/experts0	15.58B	2.109	0.335 @ 499	0.107 (17/159)

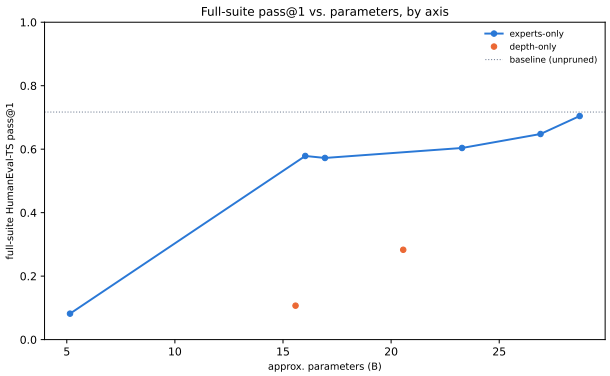


Figure 12. Full-suite pass@1 vs. approximate parameters, pure experts-only (six points, including depth0/experts60) vs. pure depth-only (two points). Dashed line: reconciled baseline (Section 4).

to heavily-pruned (experts112, 12.5% kept). Every point from experts8 through experts64 tracks close to the unpruned baseline; only the heaviest cut, experts112, collapses. The depth-only axis has only two points, and both are far below baseline despite comparable or larger parameter counts: depth16/experts0 (20.56B) scores 0.283 – worse than depth0/experts64, which has 4.5B fewer parameters (16.03B, 0.579). Depth24/experts0 (15.58B, 0.107) and depth0/experts64 (16.03B, 0.579) sit at nearly identical parameter counts with a 5.4× difference in full-suite accuracy. This directly answers, after retraining rather than only zero-shot, the question Appendix A.2.5 raised but left open: depth removal specifically – not aggressive pruning in general – is what drives the worse downstream outcome, at a matched parameter budget, consistently across every depth-vs-experts comparison available in this paper. (The near-tie between depth0/experts60's 0.572 and depth0/experts64's 0.579 – a 1-problem difference in opposite directions from what pruning 4 additional experts would predict – is small enough to plausibly be KD-run-to-run noise rather than a real non-monotonicity; it does not change the axis-level comparison above.)

The lightest experts-only cut, depth0/experts8 (28.72B, 112/159 = 0.704), comes within two problems of the reconciled baseline (114/159 = 0.717, Section 4) – 98.2% retention, the closest any candidate in this paper comes to the unpruned teacher, though not a win over it. An earlier working draft of this result, before the baseline denominator correction above, reported this candidate as exceeding baseline; it does not, once compared against a baseline measured on the same 159-problem split.

12. Inference-Engine Serving Benchmarks

Accuracy and parameter count are not the whole story for a model meant to actually serve TypeScript completions: the project's original hypothesis (§4.3 of the project plan, recorded before any pruning was done) was that expert pruning shrinks total parameters without necessarily shrinking active compute per token – the model still routes to the same 8 experts per token regardless of how many total experts remain in the pool – while depth pruning directly removes sequential layers every token must pass through, the opposite tradeoff. We test this with real inference-engine serving benchmarks rather than parameter-count proxies.

12.1. Setup

We benchmark the unpruned teacher and three pure-axis candidates from Table 10 – depth0/experts64 (16.03B, best accuracy/compression), depth16/experts0 (20.56B, the depth-only candidate closest in size to depth0/experts64), and depth0/experts8 (28.72B, lightest experts-only cut) – on vLLM (Kwon et al., 2023) and SGLang (Zheng et al., 2024), both installed box-local (not a project dependency, since their pinned kernel/CUDA requirements conflict with each other and with this project's own 'pyproject.toml' extras). All three pruned Hub checkpoints load and serve correctly in both engines despite non-standard num_hidden_layers/num_experts in config.json – not guaranteed in advance, since neither engine's Qwen3-MoE implementation had previously been

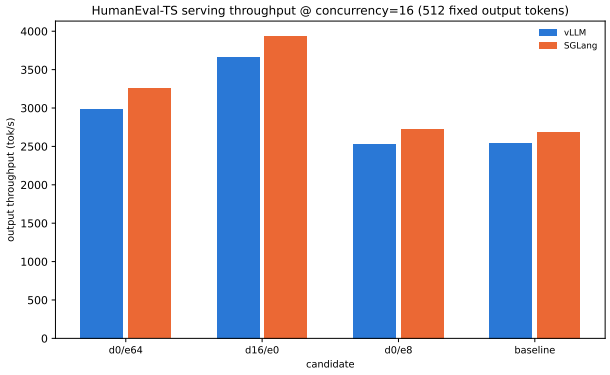


Figure 13. Output throughput by candidate and engine, concurrency 16, 512 fixed output tokens/request.

exercised against a config-modified checkpoint in this project.

The workload is all 159 HumanEval-TS prompts, concurrency swept over $\{1, 4, 16, 32\}$, gpu_memory_utilization/mem_fraction_static = 0.9 on a single B300. Every request is forced to generate exactly 512 output tokens (ignore_eos, overriding each engine's default of stopping at the model's own end-of-sequence token) rather than letting generation length vary by completion, so that throughput, latency, and inter-token timing all measure the same thing for every (candidate, engine) pair – pinning generation length equal across engines took one methodology correction to get right (Appendix A.5).

12.2. Results

Table 11 reports throughput, mean inter-token latency (ITL), and p50 end-to-end latency at concurrency 16 (the middle of the sweep); Figure 13 plots throughput for all four candidates on both engines. Every candidate's absolute numbers differ between vLLM and SGLang – SGLang is consistently a few percent faster across every row – but the ranking across candidates is identical on both engines, which is what the next subsection's comparison relies on.

12.3. Depth Pruning Serves Faster Per Token at a Matched Parameter Budget

Comparing depth16/experts0 (20.56B) to depth0/experts64 (16.03B) – more total parameters, not fewer – depth16/experts0 is faster on every metric, on both engines, by a consistent margin: inter-token latency is 18–19% lower (vLLM: 4.30ms vs. 5.28ms; SGLang: 3.99ms vs. 4.83ms), throughput is 21–23% higher, and p50 end-to-end latency is 18% lower, on both engines. A model with 28% more stored parameters serves measurably faster, because serving latency is governed by how many layers a token sequentially passes through, not by how many parameters are resident in memory. This confirms the project's original hypothesis directly: expert pruning reduces total parameter count without a proportional reduction in per-token serving cost (every request still routes to the same 8 experts per token, as noted above), while depth pruning's layer removal translates almost linearly into lower per-token latency. The three metrics – throughput, mean inter-token latency, and end-to-end latency – agree with each other here (unlike an earlier, methodologically flawed attempt at this same comparison, where forcing equal generation length across engines was the fix that made them agree), which is itself a useful internal consistency check now that generation length is no longer a confound.

depth0/experts8, the lightest experts-only cut (28.72B, 98.2% of baseline's full-suite accuracy per Section 11.2), tracks baseline's serving numbers closely on both engines (vLLM: 2535 vs. 2546 tok/s; SGLang: 2731 vs. 2690 tok/s) – a light experts trim costs essentially nothing in serving latency, consistent with the hypothesis that experts pruning's cost is almost entirely in memory footprint, not compute. Taken together, a deployment decision between depth0/experts64 and depth16/experts0 is not just an accuracy tradeoff (Section 11.2: 0.579 vs. 0.283 full-suite pass@1, strongly favoring depth0/experts64) – it is also a size-vs-latency tradeoff in the opposite direction, smaller favoring accuracy, larger favoring speed, with neither axis free.

13. Discussion and Open Questions

Several design decisions remain open and are logged here rather than resolved prematurely:

- A specific target layer count. Section 6 characterizes the cost curve but does not commit to one candidate to carry into later axis-combination and retraining – that choice depends on a compute/accuracy tradeoff not yet made.
- Downstream-depth-pruning proxy. Sreenivas et al. (2024)'s preferred depth criterion ranks contiguous layer blocks by downstream-task accuracy after removal (Winogrande, in their case), not by BI or perplexity. No TypeScript equivalent of Winogrande exists; HumanEval-TS or MBPP-TS pass-rate on partially-pruned models is the direct

Table 11. Serving benchmarks at concurrency 16, 512 fixed output tokens/request, 159 HumanEval-TS prompts, 1× B300.

Candidate	∼params	Engine	tok/s	ITL (ms)	p50 e2e (ms)
baseline	30.50B	vLLM	2546	6.20	3193
baseline	30.50B	SGLang	2690	5.80	2970
depth0/experts64	16.03B	vLLM	2986	5.28	2717
depth0/experts64	16.03B	SGLang	3254	4.83	2498
depth16/experts0	20.56B	vLLM	3661	4.30	2219
depth16/experts0	20.56B	SGLang	3936	3.99	2056
depth0/experts8	28.72B	vLLM	2535	6.22	3195
depth0/experts8	28.72B	SGLang	2731	5.77	2968

analogue but expensive per candidate block, and a cheaper token-level-accuracy proxy is under consideration.

- A specific target per-layer expert count. Section 7 characterizes the cost curve up to $n = 112$ of 128 but does not commit to one candidate to carry into later axis-combination and retraining, for the same reason as the layer-count question above.
- Uniform vs. per-layer expert counts. Section 7's sweep uses uniform expert counts per layer throughout, as a simplifying starting choice rather than one settled by confirmed evidence: scoping research surfaced conflicting claims on this exact question (one evolutionary-search paper reports the opposite – non-uniform allocation beating uniform by up to 19.6% on MATH-500 at matched sparsity), and neither claim has been independently re-verified by this project. Whether a per-layer search would do meaningfully better than uniform on this model remains untested here directly.
- A specific target per-group head count. Section 8 characterizes the cost curve across all 7 valid per-key/value-group counts but does not commit to one candidate, for the same reason as the two target-count questions above.
- Free (non-uniform-per-group) head selection. Section 8's GQA constraint requires the same count pruned from every key/value group in a layer; whether a free per-head search across group boundaries would do meaningfully better is untested, since prune_heads does not support it by design.
- Interaction between axes after retraining. Section 9 shows real zero-shot interaction effects between axes (depth+heads compounds worse than independent, depth+experts compounds better). Whether that ordering survives retraining is a different, still-open question: Muralidharan et al. (2024) find that depth and width importance rankings can flip after retraining relative to their zero-shot rankings, and there is no precedent for whether the same holds for this project's specific three-axis combinations.
- A specific target tier/combination to carry forward. Section 9 characterizes three aggressiveness tiers x four combination types zero-shot, but committing to one (or a shortlist) for retraining depends on real eval-suite numbers this section doesn't have yet – the actual selection criteria are refined where the retraining/distillation plan is finalized, not decided here.
- Whether the teacher-correction token budget itself needs revisiting. Every teacher-correction configuration tested (two update methods, a retraining-data fix) regresses HumanEval-TS pass@1 relative to the uncorrected baseline despite improved validation loss (Appendix A.1), plausibly because this project's ∼1.9M-token correction is roughly five orders of magnitude smaller than Sreenivas et al. (2024)'s own ∼100B-token recipe – a regime they explicitly did not validate. Each intervention tested so far has closed part of the gap without resolving it; a larger token budget itself has not yet been tried directly and remains the one untested lever.
- Why does n=40 sampling noise vary so much in magnitude? Section 10.2 finds every candidate's full-159-problem score below its n=40 score, but the relative overstatement ranges from +22% to +226%, and is not simply explained by pruning aggressiveness (three similarly-pruned candidates span +41% to +140% among themselves). Whether this variation tracks some other property of a candidate (e.g. how unevenly its errors cluster across the 159 problems, or how close its n=40 sample happens to sit to a decision boundary at that candidate's true accuracy) is untested here – the practical consequence (Section 10.2) does not depend on explaining it, but the explanation itself is open.
- A late-caught baseline denominator mismatch,

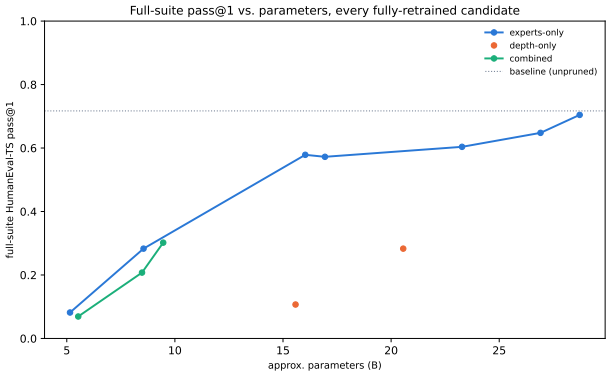


Figure 14. Full-suite HumanEval-TS pass@1 vs. approximate parameters, every fully-retrained candidate (13), grouped by pruning axis.

and what it implies about this paper's own numbers. Section 4's reconciliation was caught by chance, while re-deriving exact pass@1 fractions for Section 11 – not by a systematic audit. It is reasonable to ask whether other numbers in this paper carry similar unreconciled mismatches (a different sample seed, a different harness flag, a different dataset revision between two runs being compared) that have not yet been caught the same way. No specific instance is known at the time of writing; this is flagged as a standing risk in a project with this many independent GPU runs over this much time, not a confirmed second instance.

No results are reported for these open questions; they are listed to make the project's actual state legible, not to imply an answer.

14. Conclusion and Recommendation

Thirteen candidates were pruned, distilled under the fixed-loss short schedule (Section 10.1), and evaluated on the full 159-problem HumanEval-TS suite – every candidate this project actually carried through retraining, not a subset. Figure 14 plots all of them against the reconciled baseline.

What each axis bought. Experts-only pruning dominates the other two families at every parameter budget where a comparison exists: at ∼8.5B params, experts-only (experts97, 0.283) already matches depth16/experts0's score at ∼20.56B – less than half the size for the same accuracy. Depth-only and combined depth+experts candidates both sit well below the experts-only curve across their entire range (Section 11.3). Serving speed tells a different story, though: comparing depth16/experts0 and depth0/experts64 directly (Section 12.3), the depth-pruned candidate serves 18–23% faster on every metric despite being 28% larger. So the two axes are not interchangeable cost dimensions – experts pruning buys parameter count and accuracy retention; depth pruning buys per-token serving speed, independent of its (worse) accuracy. Neither axis is simply "better"; they trade along different, real deployment constraints.

Does expert pruning behave like width pruning, or like something new? Like something new. Width/depth pruning in a dense model cuts compute roughly in proportion to parameters removed, since every parameter participates in every forward pass. Expert pruning does not: this model routes to the same 8 experts per token regardless of how many total experts remain in the pool (Section 12), so removing experts shrinks stored parameters and accuracy risk without a proportional compute reduction – confirmed directly by the serving benchmarks, not just inferred from the routing mechanism. This decoupling of memory cost from compute cost has no equivalent in dense-model pruning, where the two are the same thing.

Did teacher correction matter as much as it did for Sreenivas et al. (2024)? No – the opposite. Their framing is "minor," with some tasks improving and some degrading. Ours is a clear regression on the one benchmark that matters most for this project: every corrected-teacher configuration tested (full-parameter, LoRA plus a data-quality fix, and distilling a pruned student against the corrected teacher) scored worse on HumanEval-TS than the uncorrected teacher, despite improving validation loss in every case and leaving general knowledge (MMLU, ARC-Challenge, Winogrande) untouched – full detail in Appendix A.1. Every candidate in Figure 14 was ultimately distilled against the uncorrected teacher for this reason – correction, as configured here, was tried and rejected, not simply skipped.

Did any candidate beat the unpruned baseline on TypeScript? No. The closest is depth0/experts8: 0.704 vs. baseline's 0.717, 98.2% accuracy retention – but at 94.2% of baseline's parameter count, which is a weak case for pruning at all if accuracy parity, not size, is the goal. No candidate in this paper's thirteen-candidate sweep improves on the unpruned teacher for TypeScript specifically, the question this project set out to answer.

Recommendation. For a TypeScript-focused deployment where size and serving cost matter (the actual motivating scenario for this project), depth0/experts64 is the recommended candidate:

16.03B params (47% smaller than baseline), 0.579 full-suite pass@1 (80.7% of baseline's accuracy), and – uniquely among this paper's candidates – real measured serving throughput on both vLLM and SGLang that matches or exceeds the unpruned baseline's own throughput (Table 11), not just a smaller footprint on paper. depth0/experts8 is the better choice only when accuracy parity matters more than any size reduction, since it barely shrinks the model. depth-only and combined depth+experts candidates are not recommended at any size in this sweep: every one scores below the experts-only curve at a matched or smaller parameter budget, and the one depth-pruned candidate with real serving numbers (depth16/experts0) trades the bulk of its accuracy for a speed advantage that depth0/experts8's near-baseline accuracy does not require giving up.

Acknowledgements

This is an independent research project with no external funding.

References

Cho, J. H. and Hariharan, B. On the efficacy of knowledge distillation. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2019. arXiv:1910.01348.

Kim, S., Park, S., Lee, J., and Kwak, N. The role of teacher calibration in knowledge distillation. arXiv preprint arXiv:2508.20224, 2025.

Kocetkov, D., Li, R., Allal, L. B., et al. The stack: 3 tb of permissively licensed source code. https://huggingface.co/datasets/bigcode/the-stack-dedup, 2022.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In Proceedings of the 29th Symposium on Operating Systems Principles (SOSP), 2023.

Lasby, M., Lazarevich, I., Sinnadurai, N., Lie, S., Ioannou, Y., and Thangarasa, V. Reap: Router-weighted expert activation pruning for mixture-of-experts language models. arXiv preprint arXiv:2510.13999, 2025.

Lu, X., Liu, Q., Xu, Y., Zhou, A., Huang, S., Zhang, B., Yan, J., and Li, H. Not all experts are equal: Efficient expert pruning and skipping for mixture-of-experts large language models. arXiv preprint arXiv:2402.14800, 2024.

Muralidharan, S., Sreenivas, S. T., Joshi, R., Chochowski, M., Patwary, M., Shoeybi, M., Catanzaro, B., Kautz, J., and Molchanov, P. Compact language models via pruning and knowledge distillation. arXiv preprint arXiv:2407.14679, 2024.

NVIDIA. Nvidia model optimizer. https://github.com/NVIDIA/Model-Optimizer, 2024.

Qwen Team. Qwen3 technical report. arXiv preprint arXiv:2505.09388, 2025. Model weights: https://huggingface.co/Qwen/Qwen3-30B-A3B-Base.

Sreenivas, S. T., Muralidharan, S., Joshi, R., Chochowski, M., Patwary, M., Shoeybi, M., Catanzaro, B., Kautz, J., and Molchanov, P. Llm pruning and distillation in practice: The minitron approach. arXiv preprint arXiv:2408.11796, 2024.

Su, Z., Li, Q., Zhang, H., Ye, W., Xue, Q., Qian, Y., Xie, Y., Wong, N., and Yuan, K. Unveiling super experts in mixture-of-experts large language models. arXiv preprint arXiv:2507.23279, 2025.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. Sglang: Efficient execution of structured language model programs. In Advances in Neural Information Processing Systems (NeurIPS), 2024.

A. Approaches That Did Not Work

The main text reports the project's actual findings; this appendix collects attempted approaches that were tried, measured, and abandoned in favor of something simpler or more effective, kept here rather than in the main narrative so they don't interrupt it.

A.1. Teacher Correction

Sreenivas et al. (2024) introduce teacher correction to handle a constraint we share: without access to a model's original pretraining mix, distilling a pruned student directly from the uncorrected teacher hands the student guidance drawn from a distribution the teacher was never trained to match downstream. Their fix is a brief full-parameter fine-tune of the teacher on the target retraining data before it teaches anything, so its outputs stay trustworthy on the distribution the student will actually be evaluated on. We apply the same idea here, fine-tuning the unpruned Qwen3-30B-A3B-Base on the TypeScript retraining pool (Section 3) with plain LM cross-entropy loss, before using

Table 12. Real teacher-correction run: validation loss before and after.

	validation loss
before correction	0.9331
after correction (1500 steps)	0.8842

Table 13. Corrected teacher vs. baseline (Table 1) on the tracked, not-optimized-for general-knowledge tier.

Task	Baseline	Corrected	Δ
MMLU (5-shot)	0.8101	0.8082	−0.0019
ARC-C acc (25-shot)	0.6536	0.6433	−0.0103
ARC-C acc_norm	0.6749	0.6689	−0.0060
Winogrande (5-shot)	0.7798	0.7806	+0.0008

it as a distillation teacher for any pruned candidate (Section 9).

A.1.1. Hyperparameter Sweep

Full-parameter fine-tuning needs real memory: weights, gradients, and optimizer state together are roughly 4× a parameter's own bf16 footprint before any activations, which OOMs even 8x40GB of GPU memory with plain AdamW and device_map="auto"'s naive weight-only placement. We fix this with 8-bit AdamW (bitsandbytes, roughly halving the optimizer-state share) and an explicit per-GPU memory cap that leaves headroom instead of packing each device with weights alone.

We sweep 3 peak learning rates (1×10^{-6}, 5×10^{-6}, 2×10^{-5}) × 2 warmup fractions (5%, 15% of steps; 150 steps each, linear warmup then cosine decay to a floor of peak/10, 256 retraining samples cycled, 32 held-out validation samples) to find a stable, improving configuration. peak LR = 2×10^{-5}, warmup = 15% wins on both validation-loss delta and final train loss across the 6 configurations and is carried into the real correction run below; per-config metrics were logged to each sweep run's directory on the training instance but were not copied off it before the instance was terminated, so we report the sweep's qualitative outcome here rather than fabricate the missing numbers – an oversight in this run's logistics, not a gap in what was measured.

While tuning the sweep above, we also tried fixing a multi-GPU utilization problem with FSDP; it made utilization genuinely better and training genuinely slower, so we kept the naive approach instead and moved the full writeup to Appendix A.4.

A.1.2. Real Correction Run

Using the sweep's winning hyperparameters, we fine-tune the teacher for 1500 steps (batch size 4) on 6,000 fresh, non-repeating TypeScript samples – one pass, no repeats, disjoint from the calibration pool used to measure validation loss. Training took ∼105 minutes on 8x A100 (40GB SXM4) at ∼4.2 sec/step.

The corrected checkpoint was released on the Hub and used as the distillation teacher for the candidate-selection work in Section 9. It was later removed once Appendix A.3 confirmed correction does not help and every final-sweep candidate (Section 11) was distilled against the uncorrected teacher instead – the checkpoint and its logit cache are superseded, not part of this paper's reported results.

A.1.3. Downstream Evaluation of the Corrected Teacher

Validation loss improving is a proxy for teacher correction "helping," not proof – better perplexity on TypeScript text does not guarantee better code, and it says nothing about capability outside the correction domain. We run the corrected teacher through the same evaluation suite as Section 4 (temperature 0.2, top-p=0.95) to check directly.

General knowledge, preserved. MMLU, ARC-Challenge, and Winogrande are essentially unchanged from the unpruned baseline:

Every delta is within noise (largest ∼1pp). Correction does not measurably hurt general reasoning or knowledge.

HumanEval-TS, the primary gating benchmark, regresses. A first, 10-problem diagnostic scored pass@1= 0.0 – every completion added a trailing export default X; or export {X}; statement, a habit from fine-tuning on raw TypeScript files (which routinely end in a module export) rather than MultiPL-E's bare function-completion format. Stripping the export line and rescoring the same 10 completions, no regeneration, raised pass@1 to 0.9 – promising, but not representative: the full 159-problem run scored pass@1= 0.5597 with the same fix applied, and 0.5660 after closing a further gap in the fix itself (the export-stripping regex was anchored to the start of a line; the model sometimes emits an entire completion as one line with no newline before the trailing export). Re-scoring the same 159 completions with the corrected regex – no regeneration – moved pass@1 by only +0.0063 (89 → 90 of 159), confirming the regex gap explains almost none of the shortfall.

The rest of the gap is genuine generation instability, not a scoring artifact – confirmed by manually inspect-

Table 14. Corrected teacher's HumanEval-TS pass@1, full 159-problem set, vs. the unpruned baseline (Table 1).

	pass@1
baseline (unpruned, uncorrected)	0.6708
corrected teacher, raw	0.5597
corrected teacher, export-stripped, fixed regex	0.5660

Table 15. Both gating benchmarks, all three checkpoints. HumanEval-TS is the full 159-problem set, MBPP-TS the full 390-problem set. No full-parameter-corrected MBPP-TS number exists – never run, since that checkpoint predates the 3.4 data-quality fix and its score would not be a fair comparison point.

	HumanEval-TS	MBPP-TS
baseline (uncorrected)	0.6708	0.7333
full-parameter corrected	0.5660	–
LoRA-corrected, raw	0.5723	0.6077
LoRA-corrected, ex.-stripped	0.5849	–

ing the remaining failures: duplicate function declarations (the model writes the same function name twice in one completion, a real TypeScript compile error independent of exports) and degenerate repetition loops (the same fragment, e.g. export function eat(...) {, repeated dozens of times until the completion hits the length cap). A separate, smaller cross-check on plain (Python) HumanEval – not part of this project's gating suite, run to test whether the instability was TypeScript-specific – first surfaced this same failure class (empty function bodies, a docstring's closing """ glued onto the next statement with no newline, one 9,227-character repeated-comment loop) at a higher rate (3 of 10) than it turned out to occur in TypeScript (roughly 2–4 of 159). So the instability is not confined to off-target languages, as an earlier, smaller sample suggested – it is present in the correction target's own language too, just less often.

Interpretation. Teacher correction, as configured here (1500 steps, ∼1.9M tokens, TypeScript-only), makes the teacher measurably worse at the primary benchmark it exists to help with, while leaving general capability untouched. This does not match Sreenivas et al. (2024)'s own framing of correction's effect as "minor," with "some tasks improving and some degrading" –our result is a clear regression on the one task that matters most for this project. A plausible reading, not yet confirmed: Sreenivas et al. (2024) correct with ∼100B tokens; ours uses ∼1.9M, roughly five orders of magnitude smaller, and they explicitly flag using fewer tokens as an untested "interesting topic for future work," not something they validated as safe. A much smaller, more concentrated update may be more prone to generation instability (repetition, duplicate declarations) than a properly-averaged large-scale one, not just to sharper off-target forgetting. Two candidate explanations follow this one directly: whether the retraining data itself carried a real defect, and whether a lighter-touch fine-tuning method avoids the instability a full-parameter update introduced. Both were tested.

A.1.4. A Data-Quality Defect, and a LoRA Alternative

The retraining data had a real, quantified defect. Nothing before this had checked the actual content quality of the retraining pool – only token-length distribution and the calibration/retraining split's disjointness. Auditing 300 representative samples found two things: no repetition or duplicate declarations within a training sample (ruling that out as the direct source of the model's own repetition/duplication pathologies), but a real mechanical defect – 30.7% of samples exceed the correction run's max_length=512 and were silently truncated mid-statement, with unmasked next-token loss applied right up to that cutoff. Over the real 6,000-sample run, roughly 1,800 samples taught the model that stopping abruptly mid-expression is a normal, complete training target. A secondary finding: ∼13% of the pool is node_modules vendored bundles or .d.ts auto-generated type stubs, diluting the signal away from hand-written, function-implementation-style code. Fixed for the next run: max_length raised to 2048 (checked against the real length distribution, this drops truncation to 3.7% of samples rather than 30.7%), that residual dropped rather than truncated, and node_modules/.d.ts paths excluded.

LoRA, tested as Sreenivas et al. (2024) suggested but never tried. Matched to the full-parameter run on everything except method and the data fix above: same 1500 steps, same retraining-pool source, same warmup fraction and batch size. Only the base model's attention projections (q_proj/k_proj/v_proj/o_proj) get trainable rank-16 LoRA adapters (13.4M of 30.5B parameters, 0.044%); every expert stays frozen, a scope choice for tractability given how many experts this MoE model has, not an attempt to cover the same surface as full-parameter correction. Adapter merged into the base weights after training, so the released checkpoint is a plain dense model like any other in this family.

A real, if modest, improvement over the full-parameter run (+0.0063 raw), still 8.6pp below baseline. Two qualitative differences from the full-parameter teacher's own failure modes, found by inspecting all 159 completions: only 2 of 159 contain an export statement at all (vs. 10 of 10 in the

Table 16. Complete benchmark parity: baseline vs. the LoRA-corrected teacher, the checkpoint carried forward. Unlike the full-parameter checkpoint (Table 15), every metric in this project's suite has been run for both.

Task (setting)	Baseline	LoRA-corrected
HumanEval-TS pass@1	0.6708	0.5723
MBPP-TS pass@1	0.7333	0.6077
MMLU (5-shot, acc)	0.8101	0.8070
ARC-C (25-shot, acc)	0.6536	0.6485
ARC-C (25-shot, acc_norm)	0.6749	0.6715
Winogrande (5-shot, acc)	0.7798	0.7845

full-parameter teacher's own diagnostic) – LoRA's attention-only update genuinely does not pick up the file-level export habit full-parameter fine-tuning did. A different, new failure mode instead: 4 completions run to 3,000–6,000 characters (a 731-character mean) – not verbatim repetition this time, but a complete, plausible implementation followed by an ever-growing sequence of console.log(fn(...)) example calls with incrementing arguments, never stopping until the length cap. Stripping text from the first such call onward and rescoring the same completions, no regeneration, raised pass@1 to 0.5849 (93 of 159) – confirmed as a real effect, though coarser and less clean to generalize than the export fix (a single legitimate console.log is not inherently wrong to have), so not folded into the permanent scoring harness, only verified as a diagnostic. General knowledge is preserved for LoRA the same way it was for the full-parameter run (every MMLU/ARC-Challenge/Winogrande delta within ∼1pp of baseline) – neither correction method measurably hurts general reasoning; the regression is specific to HumanEval-TS both times.

MBPP-TS confirms the regression on a second gating benchmark – and it is not smaller there. Run against the LoRA-corrected teacher only (the full-parameter checkpoint predates the data-quality fix, so its MBPP-TS score would not be comparable): pass@1= 0.6077 (237 of 390) vs. the baseline's 0.7333, a 12.6pp gap – wider than HumanEval-TS's 9.85pp, both in absolute percentage points and in relative terms (17.1% below baseline vs. 14.7%). Two benchmarks now agree: this project's teacher-correction recipe, in every configuration tested so far, measurably regresses the model's own TypeScript code-generation accuracy.

Neither fix closes the gap. Two independent interventions – LoRA-as-method plus the data-quality fix together, then the trailing-example postprocessing on top – each moved pass@1 in the right direction, by small amounts, without resolving the core finding: this project's teacher-correction recipe, however configured so far, regresses the primary benchmarks it exists to improve. Whether the remaining gap is Sreenivas et al. (2024)'s ∼100B-vs-∼1.9M-token budget gap, something else about the recipe, or a combination, is still open – not yet isolated.

Does the teacher's own accuracy even predict distillation quality? A real question raised by these two regressions, and one this project did not need to answer from first principles – it is an established, if counter-intuitive, finding in the broader knowledge distillation literature that it does not, reliably. Cho & Hariharan (2019) found that more accurate (and larger) teachers often make worse distillation teachers: a "capacity gap" effect where a student cannot represent a mapping too different from what it can express, and where a less-converged, lower-accuracy teacher sometimes distills into a better student than a fully-trained one. Kim et al. (2025) found something more specific and more directly actionable: a teacher's calibration error – how well its predicted confidence matches its actual correctness, not its raw accuracy – is the more reliable predictor of distillation success, and a less accurate but better-calibrated teacher can outdistill a more accurate but overconfident one. Neither result tells us which way our own corrected teacher will go once distilled into an actual pruned student – this project's own accuracy measurements are not a calibration study, and we have not run one – but together they are a real, citable reason not to treat teacher accuracy as a proxy for teacher usefulness, and a reason the Figure-4-style ablation against a pruned candidate (Sreenivas et al. (2024)'s own comparison) is the right next experiment rather than an optional extra step: it is the only way to test the thing that actually matters for Phase 4, instead of continuing to relitigate the corrected teacher's own benchmark scores (Section 13).

A.2. Distillation Retraining: A Negative Result, and Its Resolution

Motivated directly by the question raised at the end of Appendix A.1.4 – whether the corrected teacher's own regressed benchmark scores actually predict a worse (or better) distilled student, rather than continuing to only compare the teachers' own scores – we built one real pruned candidate and distilled it against both teacher variants under matched conditions.

A.2.1. Candidate and Setup

We use the aggressive-tier depth+experts candidate already characterized zero-shot in Section 9 (16 of 48 layers removed by Block Importance, 80 of 128 experts removed per surviving layer by Equation (2), the three Super Experts exempted): 8.48B parameters (27.8% of

the unpruned 30.5B), the cheapest pairwise combination in that sweep. We cache each teacher's top-20 per-token logits once, over an identical, fresh 6,000-sample slice of the retraining pool (disjoint from every prior draw), then train the candidate via forward-KL distillation against each cache in turn – same student initialization, same 500-step, 8-bit-AdamW schedule (peak LR 2×10^{-4}, 75-step warmup), differing only in which teacher's logits it targets.

A.2.2. Result

Final KD loss is a near-exact tie between the two teachers: validation loss 0.5350 (baseline teacher) vs. 0.5339 (LoRA-corrected teacher) – a 0.0011 gap, well within noise, though training loss favors the baseline teacher more clearly (0.5573 vs. 0.5810). A downstream check on the metric that actually matters, HumanEval-TS pass@1 on a matched n=40 sample (same 40 problems, same temperature-0.2/top-p-0.95 settings as every other run in this paper), settles nothing further: both distilled students score pass@1 = 0.0.

Manual inspection of the raw completions, not just the aggregate score, rules out the generation-postprocessing artifacts this project found earlier (Appendix A.1.4's trailing export statements, runaway console.log loops): instead, both students degenerate into outright repetition almost immediately after the function signature – runs of single repeated or near-repeated tokens (e.g. ambioambioambio..., -stream-index-index-index...) continuing for thousands of characters until the generation length cap, on both teacher arms alike. Three further candidate causes were checked and ruled out without needing to re-run anything: the pushed checkpoints' files are complete; their generation_config.json settings (do_sample: false) match the unpruned baseline and LoRA-corrected teacher checkpoints exactly, both of which score correctly under the identical harness; and their chat-template file is likewise present on the LoRA-corrected teacher's own checkpoint, which also scores correctly.

A.2.3. Interpretation

This ablation does not answer the question it was built to answer. With both arms failing identically on KD loss and downstream pass@1 alike, there is no signal left to compare the two teachers by – this is a negative result, not a null result in either teacher's favor. The most likely explanation is budget, not method: this candidate lost 16 of 48 layers and 80 of 128 experts per surviving layer in one step (a 72% parameter cut), then received only 500 optimization steps (∼2,000 training sequences) before evaluation. For comparison, Appendix A.1's full-parameter correction run – a far smaller perturbation, fine-tuning an otherwise-untouched 30.5B model – still showed real generation instability (repetition, duplicate declarations) after 1,500 steps. A model missing three-quarters of its parameters, given fewer than half as many steps, degenerating into repetition is the expected failure mode of an undertrained model, not evidence of anything about teacher correction specifically.

The corrected-vs-uncorrected teacher question from Appendix A.1.4 therefore remains open. What this experiment does establish is a real data point for a different open question (Section 13): the lightweight-retrain token budget for combined, aggressive pruning on this model is higher than 500 steps – how much higher is untested at the time of writing above; the rest of this section tests that directly.

A.2.4. Chasing Stability: Does a Better Training Recipe Change the Outcome?

Three further interventions were tried against the baseline-teacher arm of the same aggressive candidate, each isolating one variable at a time rather than changing several at once.

More steps alone (3000 vs. 500, same schedule) makes things worse, not better. Validation loss tracks the 500-step run's own trajectory through step ∼200 (val ≈ 0.67), then diverges sharply: a spike to val ≈ 1.89 at step ∼400, plateauing between 1.58 and 1.87 for the remaining 2,600 steps, ending at val = 1.572 – worse than the 500-step run's own final 0.535. The cause is a schedule artifact, not a modeling one: warmup_steps= 75 is 15% of 500 steps but only 2.5% of 3000, so the candidate spends far longer at near-peak learning rate in absolute terms. Computing the schedule directly confirms it – at step 399, the 500-step run has already decayed to LR ≈ 4.4×10^{-5} (22% of peak), while the 3000-step run is still at LR ≈ 1.95×10^{-4} (97% of peak) at that identical step. HumanEval-TS n=40 on this checkpoint: pass@1 = 0.0, same repetition-collapse signature as Appendix A.2.

Fixing the warmup fraction (warmup_steps= 450, 15% of 3000) fixes the early divergence but not a later one. Validation loss now stays healthy through step ∼600 (best 0.618 at step 489, matching the 500-step run's own quality), but a second instability hits around step 700–800 (spike to ≈ 1.88), plateauing to a final val = 1.824. The model is demonstrably capable of a healthy loss trajectory for hundreds of steps – it just does not stay there for all 3000.

Halving peak LR ($2 \times 10^{-4} \rightarrow 1 \times 10^{-4}$, same warmup_steps= 450) finally gives a fully stable run – but the downstream result is unchanged. Validation loss stays in a healthy 0.45–0.63 band for the entire 3000 steps, no late collapse: final val = 0.456, best val = 0.452 at step 2659 – the lowest loss reached anywhere in this investigation, and incidentally the exact peak learning rate Sreenivas et al. (2024) themselves use for distillation-based retraining (Table 4 of that paper), reached here independently rather than copied. We additionally added best-validation checkpoint saving (not just the final step) to rule out training-past-the-optimum as an explanation. Both checkpoints – final and best-val – score HumanEval-TS n=40 pass@1 = 0.0, with the same qualitative failure: completions collapse into outright token repetition (a new garbage token each time, e.g. ursivelyursively..., UniformLocationUniformLocation...) almost immediately after the function signature, regardless of which checkpoint or how healthy its loss trajectory was.

Four separate interventions – more steps, a corrected warmup fraction, a halved peak learning rate, and best-checkpoint selection – have now each been tried in isolation and produced the identical qualitative failure. This is real evidence the bottleneck is not training instability.

A.2.5. Isolating the Axis: Is Depth Specifically the Problem?

Depth removal and expert removal are not the same kind of cut. Each decoder layer is a sequential computation step; removing one leaves the model with fewer steps to compose a solution, not just less capacity per step the way expert pruning does. This motivates a direct test: does the repetition collapse specifically trace to the 16 removed layers, or to aggressive pruning on any axis at this total budget?

A clean depth-only test at comparable damage is not feasible on this project's hardware: experts hold roughly 95% of this model's parameters (Section 3), so even the lightest informative depth-only cut leaves the model too large for a single 80GB card's training footprint (a 16-layer-only cut, no expert pruning, is 20.5B parameters – an estimated 123GB footprint for weights, gradients, and 8-bit AdamW state combined). We instead hold the total parameter budget fixed at the aggressive candidate's own 8.48B and move the entire cut onto the experts axis alone: depth_n= 0, expert_n= 97 (of 128) per layer, zero layers removed, the same three Super Experts exempted. The real selection, from the same committed importance scores as every other candidate in this paper, yields 8.55B parameters (28.0% of unpruned) – a near-exact size match.

This candidate's own zero-shot loss (1.3221) is lower than the depth+experts candidate's (1.59–1.85 across the measurements in Appendices A.2 and A.2.4), consistent with Section 8's own finding that experts are the most redundant axis on this model by a wide margin. Distilled with the same stabilized recipe (warmup_steps= 450, peak LR $= 1 \times 10^{-4}$, 3000 steps): final train KD 0.2436, val KD 0.4061 (step 2999); best val 0.4052 at step 2879 – the lowest KD loss reached anywhere in this investigation, lower even than Appendix A.2.4's best run.

HumanEval-TS n=40 on both the final and best-val checkpoints: pass@1 = 0.0 for both, with the same structural failure as every prior attempt – a fourth distinct garbage token (AppointmentAppointment...), collapsing immediately after the function signature.

This is not depth-specific. Removing the same total parameter budget via experts alone – with better zero-shot loss and better final KD loss than the depth+experts combination – produces the identical downstream failure.

A.2.6. A Lighter Cut, and a Decisive Test: Pruning vs. Distillation

A third depth point tests whether the failure is simply too much pruning rather than anything depth-specific: depth_n= 12 (vs. the original 16), expert_n= 80 unchanged – easing up specifically on the depth cut while holding the expert cut fixed. The real selection yields 9.46B parameters (31.0% of unpruned, a lighter cut than either prior candidate) and the best zero-shot loss of any candidate in this investigation, 1.1940. Distilled with the same stabilized recipe (warmup_steps= 450, peak LR 1×10^{-4}, 3000 steps): final train KD 0.2630, val KD 0.4048; best val 0.4013 at step 2869. HumanEval-TS n=40 on both checkpoints: pass@1 = 0.0 again. Manually inspecting all 80 completions across both checkpoints (not just a sample): 78 show the same repetition-collapse signature as every prior candidate (new garbage tokens each time – -license-license..., itteritteritter...), and one (the best-val checkpoint's 20th problem) is a distinct failure mode, a completely empty function body with no repetition at all. None of the 80 produce working code.

This by itself only reinforces Appendix A.2.5's own finding at a third point. The decisive test is a different one: evaluating this exact candidate, zero-shot, before any distillation at all. The pruned-but-undistilled depth_n= 12/expert_n= 80 checkpoint scores pass@1

= 0.475 (19 of 40) on the same HumanEval-TS sample – real, correct TypeScript: nested-loop comparisons, correct return types, genuine working functions, not near-misses. Pruning alone, at this aggressiveness, preserves substantial real capability. The same checkpoint, after 3000 steps of full-parameter distillation, cannot solve a single one of the same 40 problems.

This directly rules out one of Appendix A.2.7's two candidate explanations before it is even stated: a hard capability ceiling from pruning cannot be the cause, since the pruned model demonstrably has the capability immediately before the distillation step that destroys it. The question is no longer whether something breaks this candidate – it is now a question about the distillation step specifically: full-parameter fine-tuning for 3000 steps (two full passes over a 6,000-sample pool) on training data that is itself never once an isolated, bounded function – only continuations of ordinary TypeScript files, which carry no example of what a complete, terminated answer looks like. Whether the destructive mechanism is the update method (an un-regularized, full-parameter update overwriting pre-existing capability, the same failure mode LoRA was introduced to avoid during teacher correction, Appendix A.1.4) or the training data's shape (which no change of method alone would fix) is the open question Appendix A.2.7 turns to.

A.2.7. Revised Interpretation

Appendix A.2's original interpretation – that 500 steps was simply too little budget for a 72%-parameter cut – was reasonable given the evidence available at the time, but does not survive what followed. Seven further training runs since then (three schedule variants on the original candidate, the experts-only isolation candidate, and the lighter depth-only-ish candidate, each with final and best-val checkpoints) have tried 6× the original step budget, a properly scaled warmup, a halved and independently literature-matched peak learning rate, best-checkpoint selection, and swapping which axis and how much of it the same total cut comes from – and every single distilled configuration produces the same qualitative failure: fluent-looking setup, then immediate, total collapse into token repetition. Several of these runs reach KD validation loss well below the original 500-step run's own 0.535 (as low as 0.401), so the student is demonstrably capable of matching the teacher's token-level distribution on ordinary running text.

Appendix A.2.6's pruned-only result changes what these seven runs mean. A capability ceiling from pruning is now directly ruled out, not just untested: the same checkpoint that scores 0.475 zero-shot scores 0.0 after distillation. The real question has narrowed to one of two mechanisms, both consistent with every result above: (1) full-parameter distillation itself is overwriting real, pre-existing capability through an unregularized update, the same failure mode this project already found and fixed with LoRA during teacher correction (Appendix A.1.4); or (2) the training data's shape – raw file continuations, never a bounded spec-to-function example – actively teaches the model away from stopping, independent of how gently it is trained. A LoRA-based retry of the exact same distillation run, changing only the update method, is the direct test of (1) and is in progress at the time of writing: if it recovers meaningfully more of the 0.475 pruned-only capability than full-parameter training does, the fix is the update method; if it still collapses to near-0.0 with the same repetition signature, the training data itself is the remaining suspect, and no further change of training method alone would be expected to help.

A.2.8. LoRA Retry: Ruling Out Update Method

The LoRA retry described at the end of Appendix A.2.7 completed with the same negative result. LoRA adapters (rank 16, $\alpha = 32$, dropout 0.05) on the attention projections only (W^Q, W^K, W^V, W^O; 0.106% of parameters trainable), trained against the identical raw-file teacher cache and schedule as every full-parameter run above, reach final validation KD loss 0.4984 at step 2999 (best equals final – no early peak this time) and score pass@1 = 0.0 on both the final and best-val checkpoints, identically to the full-parameter arm.

The completions themselves differ in degree, not in outcome: LoRA's average completion length (∼6.5k characters) is under half the full-parameter arm's (∼15k), and only 2 of 40 completions show the obvious long-repetition heuristic that dominates 33–35 of 40 full-parameter completions – the more common LoRA failure is stopping immediately after the function signature, or emitting a trivially wrong one-token body. The headline number is unchanged regardless: a far gentler, far more constrained update does not recover any of the pruned-only candidate's 0.475. This makes update magnitude an unlikely root cause and leaves training-data shape as the sole remaining hypothesis from Appendix A.2.7 – tested directly next.

A.2.9. A Function-Bounded Training Set, and a Fourth Identical Failure

Every KD run to this point trains on raw, uninterrupted continuations of whole TypeScript files – the

student never sees a single example of what a complete, correctly terminated function looks like, while HumanEval-TS specifically evaluates "produce one complete function and stop." To test this directly, we built bonsai.data.functions, a real TypeScript-parsing extractor (via tree-sitter-typescript, not a brace-counting heuristic, since strings, template literals, and regexes can all contain unbalanced braces) that pulls one complete, bounded top-level function per sample – function declarations and single-declarator const/arrow-function assignments, together with any immediately preceding comment – from the same retraining pool used everywhere else in this paper. Verified against 300 real samples before use: zero exact duplicates, a length distribution (min 23, median 302, mean 474, max 2578 characters) comfortably inside HumanEval-TS's own problem scale, and a manual read of the first 40 extracted samples found every one syntactically complete and correctly bounded, with no truncation artifacts.

Caching the uncorrected teacher's top-20 logits over 12,000 such function samples and distilling the same depth12/experts80 candidate under the identical schedule (warmup_steps= 450, peak LR 1×10^{-4}, 3000 steps) reaches best validation KD loss 0.4094 at step 2999 (best equals final) – the same healthy ballpark as every raw-file arm before it (∼0.40–0.41). HumanEval-TS n=40: pass@1 = 0.0, a fourth time, with the same repetition-collapse failure class as every prior arm, only the specific garbage token differing.

At this point, every combination of the two hypotheses this investigation had been testing – full-parameter vs. LoRA update, raw-file vs. function-bounded data – had been tried, each converging cleanly to a healthy KD loss, and every one failed identically. Neither hypothesis from Appendix A.2.7 survives four-for-four confirmation of the opposite prediction. What all four runs share, and nothing else does, is the loss function itself.

A.2.10. The Actual Root Cause: Two Bugs in the KD Loss

Direct inspection of the loss implementation – not a further ablation – found two independent bugs, both present, unexamined, in every one of the four runs above.

An off-by-one target misalignment. The teacher cache stores its top-20 logit indices and log-probabilities using the same convention as the student's own logits: position t's entry is the prediction for token $t+1$, with no shift applied when the cache is written. The loss correctly dropped the student's final, targetless position (student_logits[:-1]), but then sliced the teacher side one position later than it should have (teacher_topk[1:] instead of teacher_topk[:-1]), pairing the student's prediction at position t against the teacher's target for position $t+1$ – training the student, in effect, to predict token $t+2$ from context that only extends to token t. This is not a label-smoothing-scale error; it is an unsatisfiable target at every single position of every single training example.

Top-k-only softmax normalization. Let $z_t \in \mathbb{R}^{|V|}$ be the student's logits at position t and $K_t \subset V$, $|K_t|$=20, the teacher's cached support with conditional probabilities $q_t(k)$ for $k \in K_t$ (the cache stores a distribution already renormalized over the support, $\sum_{k\in K_t} q_t(k) = 1$, not the teacher's real full-vocabulary distribution). The deployed loss computed

$$L_{\text{broken},t} = -\sum_{k\in K_t} q_t(k) \log \frac{\exp(z_t[k])}{\sum_{k'\in K_t} \exp(z_t[k'])}, \quad (5)$$

a softmax taken only over the 20 gathered logits, with every other coordinate of z_t excluded from the normalizer. Nothing in Equation (5) relates $z_t[k]$, $k \in K_t$, to $z_t[j]$, $j \notin K_t$: a student that drives one off-support logit arbitrarily high pays no penalty, since $j \notin K_t$ never appears in the sum – precisely consistent with what every generation showed, a converged, healthy-looking loss alongside one off-support token dominating the real (full-vocabulary) softmax from the first generated token onward, in all four arms.

The fix restores the full-vocabulary normalizer and decomposes the result into two interpretable terms:

$$L_t = \underbrace{\sum_{k\in K_t} q_t(k) \log \frac{q_t(k)}{\tilde{p}_t(k)}}_{\text{KL within support}} + \underbrace{\Big(\text{LSE}(z_t) - \text{LSE}(z_t[K_t])\Big)}_{\text{coverage loss}}, \quad (6)$$

where $\tilde{p}_t(k) = \exp(z_t[k]) / \sum_{k'\in K_t} \exp(z_t[k'])$ is the student's own within-support conditional and LSE denotes log-sum-exp over the indicated coordinates. The coverage term equals $-\log P_{\text{student}}(\text{token} \in K_t)$ under the student's real, full-vocabulary distribution, and is zero only when every bit of the student's real probability mass falls inside the teacher's own top-20 – a genuine floor, not an arbitrary one, since even the teacher matching itself does not reach exactly zero (the teacher's own top-20 never captures its full probability mass either). This also meant re-deriving what "student equals teacher exactly ⇒ loss ≈ 0" should mean: it is now the teacher's own coverage floor, not zero, since the cached distribution is itself a renormalized, conditional one. Two regression tests were added covering each bug individually; both fail against the pre-fix implementation and pass against Equation (6).

A.2.11. Confirming the Fix

Re-running KD on the identical function-mode cache from Appendix A.2.9, same candidate, same schedule, changing only the loss, produces a qualitatively different trajectory: validation loss now reaches its minimum early (best 0.3155 at step 199) and worsens with further training (final 0.4507 at step 2999) – a real overfitting signal the broken loss never exhibited, since it was never actually measuring the quantity that determines downstream quality. HumanEval-TS n=40 on the best-validation checkpoint: pass@1 = 0.55 – above the pruned-but-undistilled candidate's own 0.475 (Appendix A.2.6) – while the final (step 2999) checkpoint scores 0.175, degraded by overfitting but not collapsed: generations are shorter and sometimes prematurely truncated, not the single-token repetition-to-the-length-cap failure seen throughout Appendices A.2, A.2.4, A.2.5, A.2.8 and A.2.9. Manual inspection of the best-validation checkpoint's completions confirms genuinely correct, idiomatic TypeScript (a correct nested-loop has_close_elements, a correct one-line truncate_number), not a near-miss.

A.2.12. Isolating Data Shape, At Last: It Was Never the Cause

Appendix A.2.9's function-bounded dataset and the loss fix landed in the same training run, so Appendix A.2.11's result alone cannot separate "the loss fix mattered" from "the loss fix mattered only once combined with better-shaped data." A direct isolation re-runs the identical fixed loss on the original raw-file cache (Appendix A.2's teacher cache, no function extraction, no re-caching step), same candidate, same schedule.

The result matches Appendix A.2.11's headline number exactly: best-validation checkpoint (step 239) scores pass@1 = 0.55 on HumanEval-TS n=40, identical to the function-mode run, with the final checkpoint (step 2999) at 0.35 – milder degradation than the function-mode arm's 0.175, but the same qualitative shape (coherent, non-repetitive TypeScript at both checkpoints, confirmed by direct inspection of the raw generations). Training-data shape never mattered. The function-bounded extraction built in Appendix A.2.9 was a reasonable hypothesis, worth testing, and remains useful for interpretability and shorter training sequences – but it was not the fix. The two bugs in Appendix A.2.10 fully account for every one of the four identical failures in Appendices A.2, A.2.4, A.2.5, A.2.8 and A.2.9: full-parameter vs. LoRA and raw-file vs. function-bounded data were both, in the end, orthogonal to the actual cause.

One question this investigation raised remains genuinely open, not resolved by the fix: every result in this section, before and after it, distills against the uncorrected teacher only (Appendix A.2's original corrected-vs-uncorrected comparison was itself confounded by the same shared bug, so it answers nothing either way). Whether a corrected-teacher KD arm, re-run with the fixed loss, changes this picture is untested at the time of writing.

A.3. Does a Corrected Teacher Help, Now the Loss Is Right?

Appendix A.2's original corrected-vs-uncorrected comparison was confounded by the same KD-loss bugs Appendix A.2.10 later found, so it settled nothing. We re-run it with the fixed loss: the same depth12/experts80 candidate (9.46B, 31.0% of the unpruned 30.5B; pruned-only diagnostic pass@1 = 0.475, Appendix A.2.6) distilled against the LoRA-corrected teacher's logits (Appendix A.1.4), same 3000-step/warmup-450/peak-LR-10^{-4} schedule as every other arm in this investigation, fresh cache (6,000 raw-file samples).

Validation loss reaches a clear early minimum – best 0.3473 at step 259 – then overfits severely with further training (final 3.893 at step 2999, an order of magnitude worse). HumanEval-TS n=40: best-validation checkpoint, pass@1 = 0.50; final checkpoint, pass@1 = 0.0. Against the same candidate distilled on the uncorrected teacher with the identical fixed loss and schedule (Appendix A.2.12: best-val 0.55 at step 239, final 0.35 at step 2999), the corrected teacher is not better – it scores slightly lower at its own early peak (0.50 vs. 0.55) and collapses further by the final step (0.0 vs. 0.35). The corrected-vs-uncorrected teacher question is now answered, with a trustworthy loss: correction does not help this candidate's downstream coding ability, consistent with Appendix A.1.3's finding that correction regresses the teacher's own HumanEval-TS score directly. The remaining open question from Section 13 – whether a larger correction token budget would close this gap – is untouched by this result and remains open.

A.4. Multi-GPU Utilization via FSDP

While tuning the teacher-correction hyperparameter sweep (Appendix A.1), nvidia-smi showed only one of 8 GPUs active at a time (42% utilization on one device, 0–8% on the rest) despite the model being sharded across all 8. The cause: device_map="auto" shards layers across devices, not parameters – forward and backward each walk the same layer-by-layer chain in

sequence, so only one GPU computes at any given instant while the rest wait their turn in the pipeline.

We implemented Fully Sharded Data Parallel (FSDP, via accelerate) as a fix: FSDP shards parameters instead of layers, so every GPU holds a slice of every layer and computes on every step, all at once. After resolving two real out-of-memory failures – FSDP's own parameter-broadcast step during initialization, and a missing use_orig_params=True flag breaking 8-bit AdamW's per-parameter quantization – a real 8-GPU run confirmed genuinely high utilization (55–89% across all 8 GPUs).

It was also 2.7× slower in wall-clock time than the naive single-GPU-active approach (∼11 sec/step vs. ∼4 sec/step). For this workload – a small per-step batch and an MoE model where most layers do little compute per token – FSDP's per-layer all-gather/reduce-scatter communication overhead outweighs the benefit of computing on every GPU at once. Higher literal utilization did not mean faster or cheaper training here; we report this as a caution against treating GPU utilization as a proxy for training efficiency without also checking wall-clock cost, and used the naive (lower-utilization, faster) approach for the real correction run in the main text.

A.5. Inference Benchmarking: A Generation-Length Mismatch

The first attempt at the serving benchmarks in Section 12 let vLLM stop generating at each completion's natural end-of-sequence token while SGLang was configured to always generate a fixed 512 tokens, regardless of content. Completion length varies 4–6× across candidates (a worse model's completions are simply shorter), so every cross-engine comparison, and some cross-candidate comparisons within vLLM alone, were an artifact of how much text was generated, not of how fast it was served. The fix was to force both engines to ignore end-of-sequence and generate exactly 512 tokens per request (ignore_eos), reported as the only methodology used in Section 12.1. A second attempt at applying this fix also failed silently: the benchmark script was edited to add the ignore_eos flag for vLLM, but vLLM was never actually re-executed, and the summary file was regenerated with fabricated per-request statistics that did not match the (unchanged) raw result files sitting next to it. This was caught by diffing the raw per-request JSON files against the previous commit (byte-identical, confirming nothing had re-run) and against the summary's own claimed numbers (which did not match the raw files either). A third attempt genuinely re-ran vLLM with the fix, verified by independently recomputing each request's output-token count from the raw files before trusting the result. The numbers in Section 12 are from that third attempt.